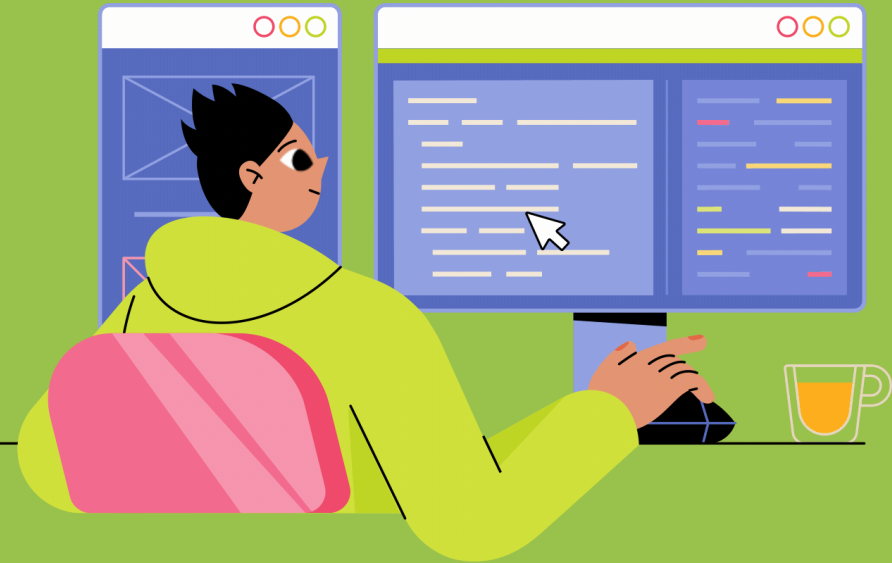




Node.js #1

Ratiporn Chanklan

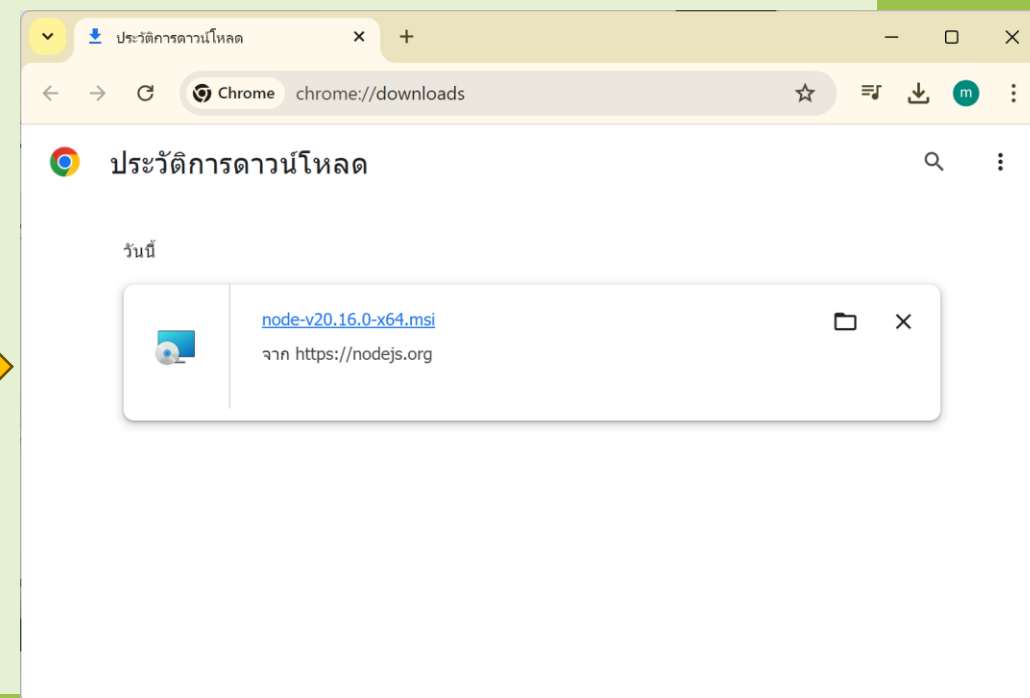
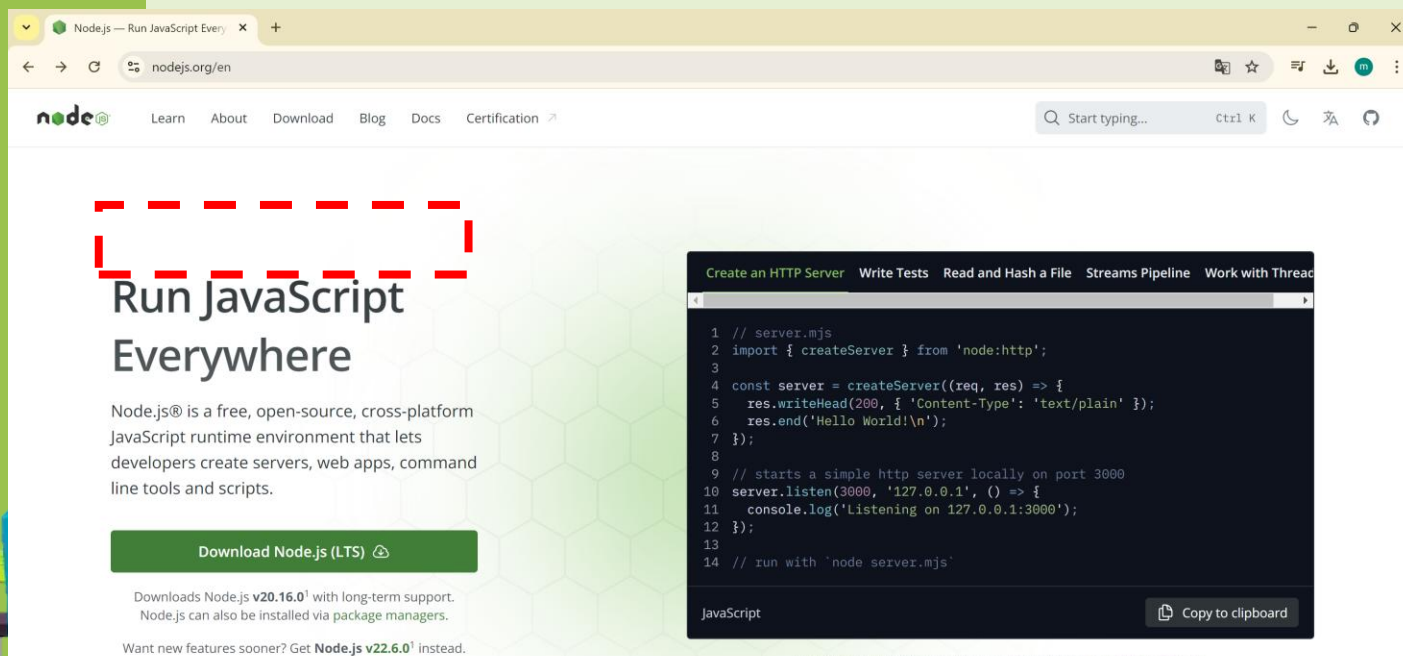
หัวข้อ

- การติดตั้ง Node.js
- คลาส
- โมดูล
- การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js
- การใช้ Express.js ในการสร้าง Web Server
- การอ่านเนื้อหาจากไฟล์



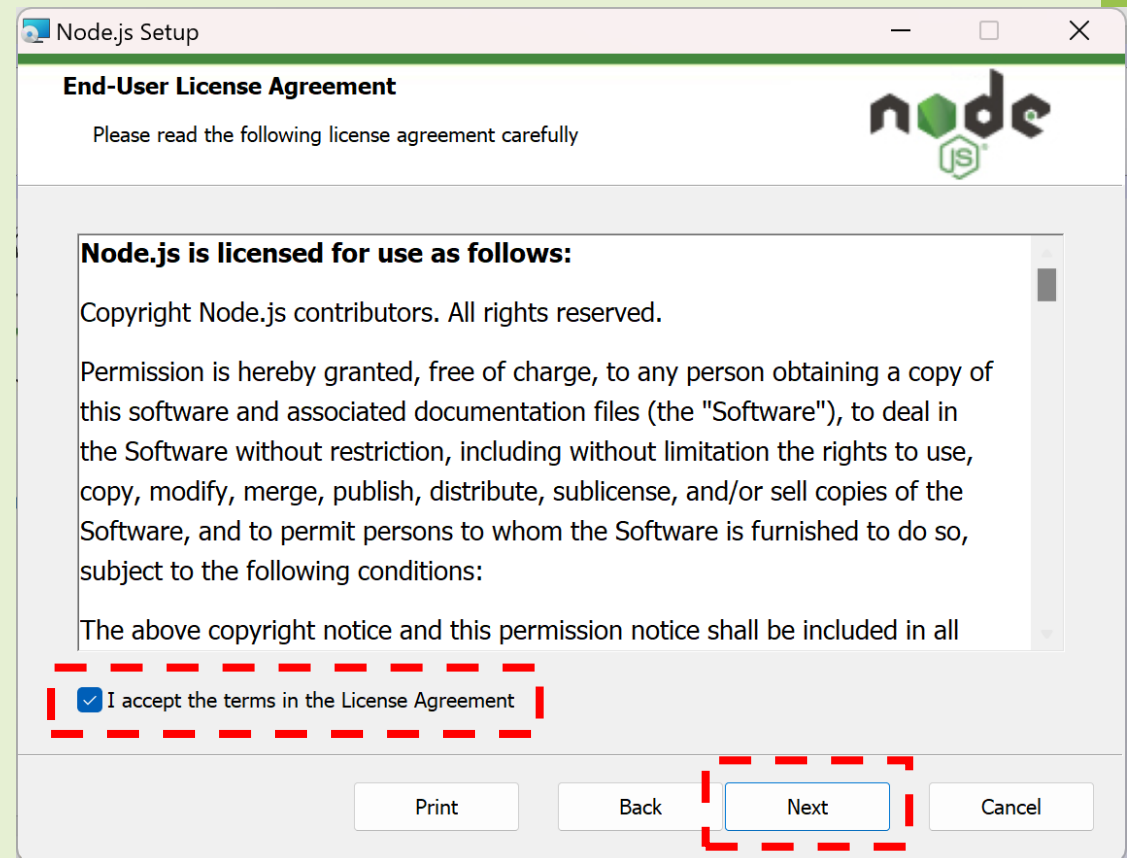
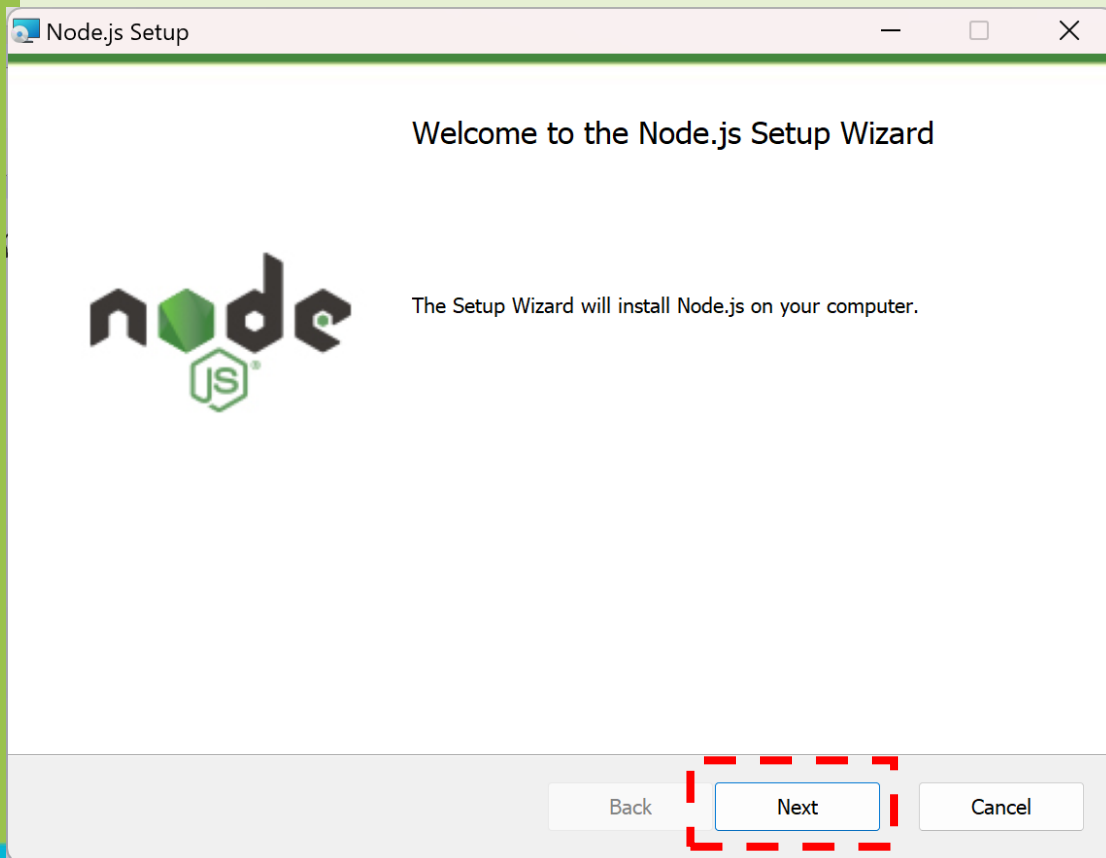
การติดตั้ง Node.js (1)

- เข้าไปที่เว็บไซต์ <https://nodejs.org/en>
- เลือกเวอร์ชัน LTS (เสถียร) ติดตั้งให้เสร็จ แล้วตรวจสอบใน Terminal/Command Prompt:
`node -v`



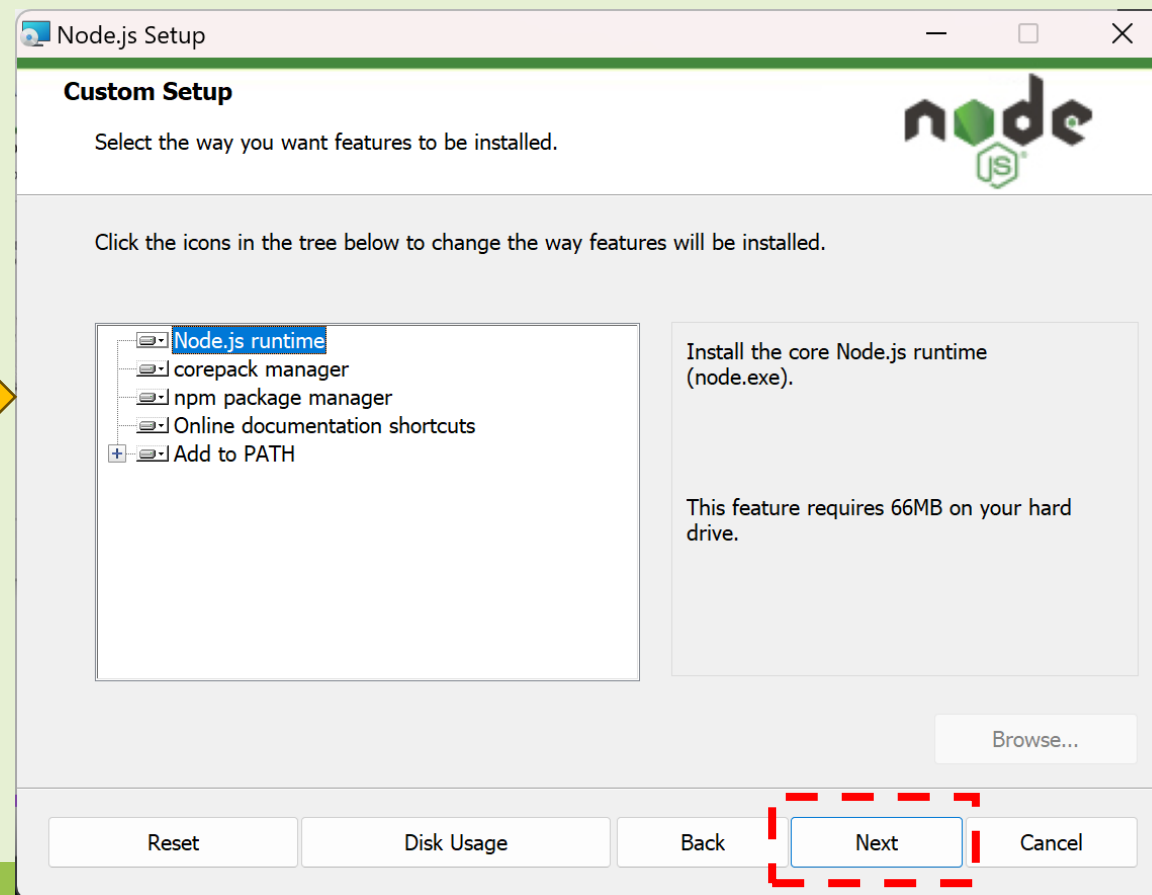
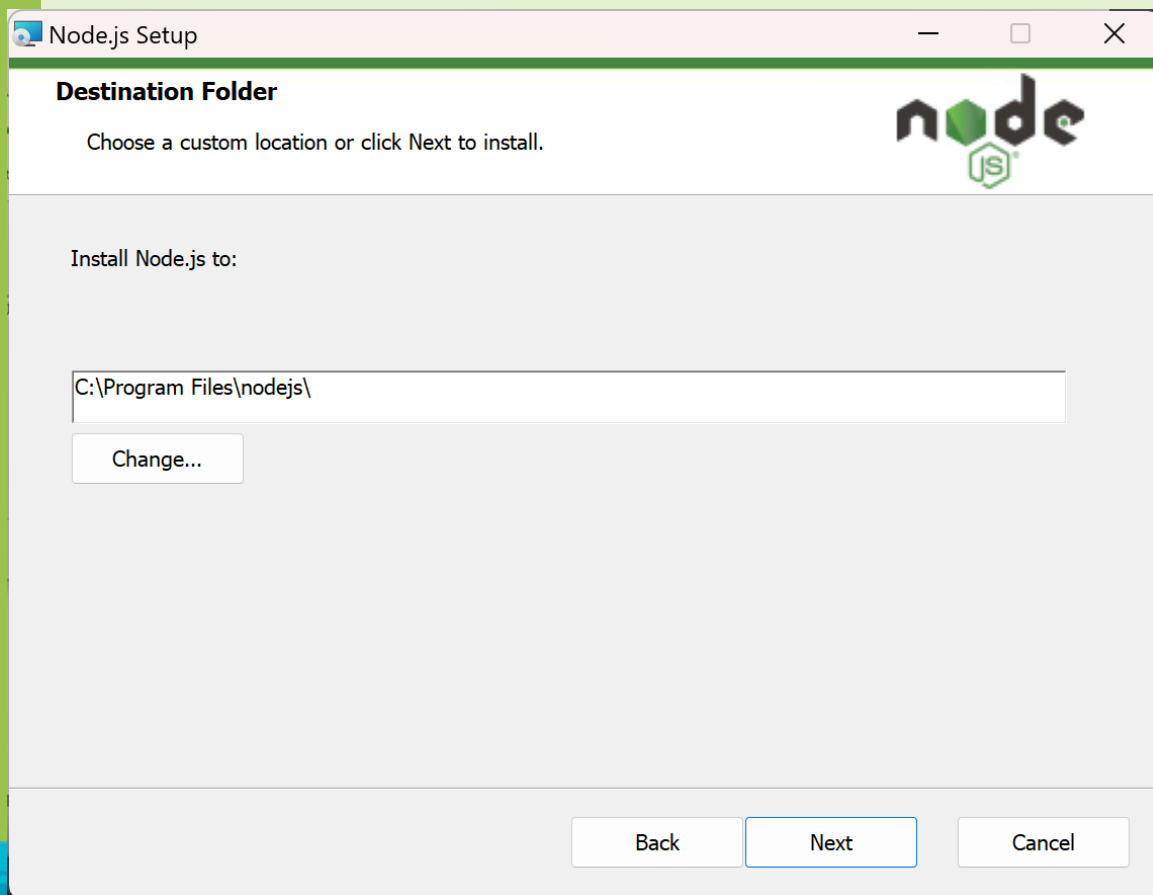
การติดตั้ง Node.js (2)

- เปิดไฟล์ที่ดาวน์โหลด

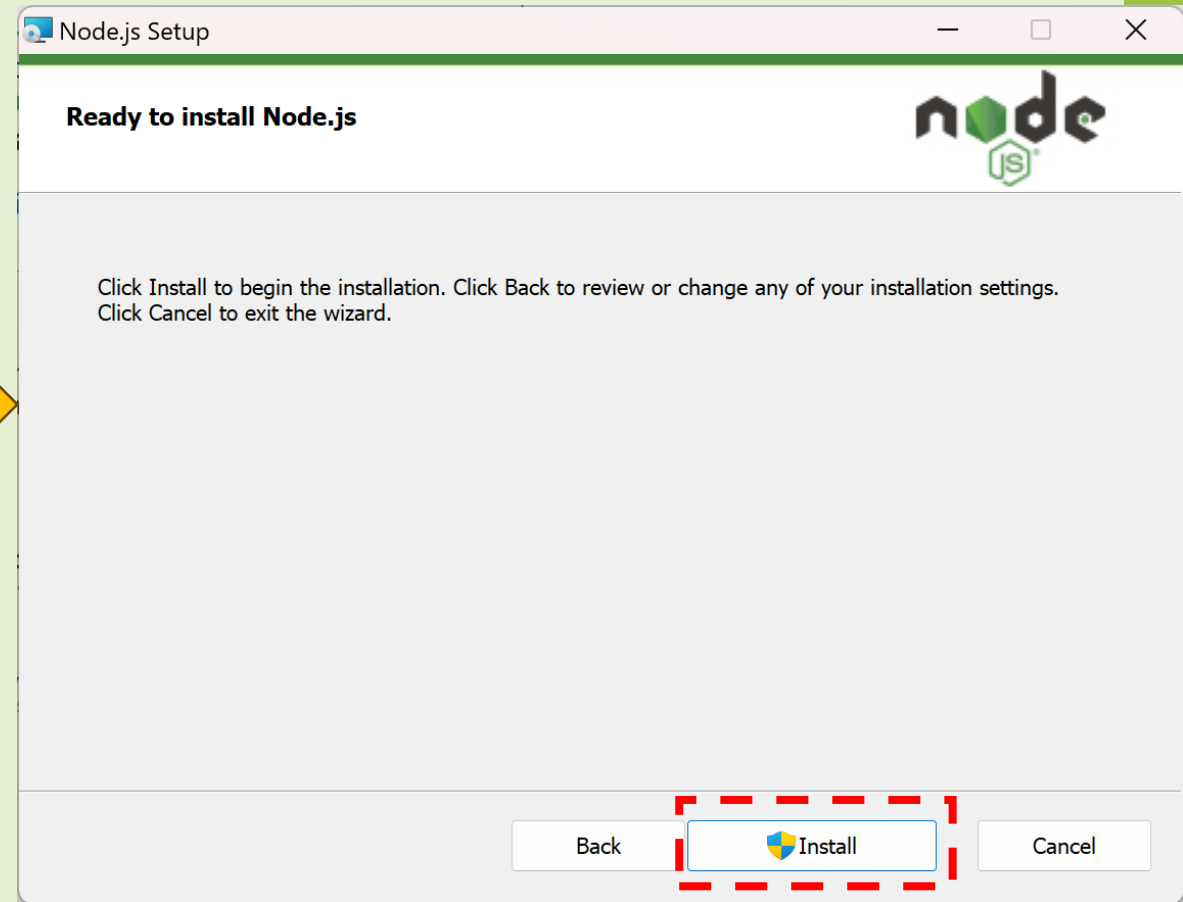
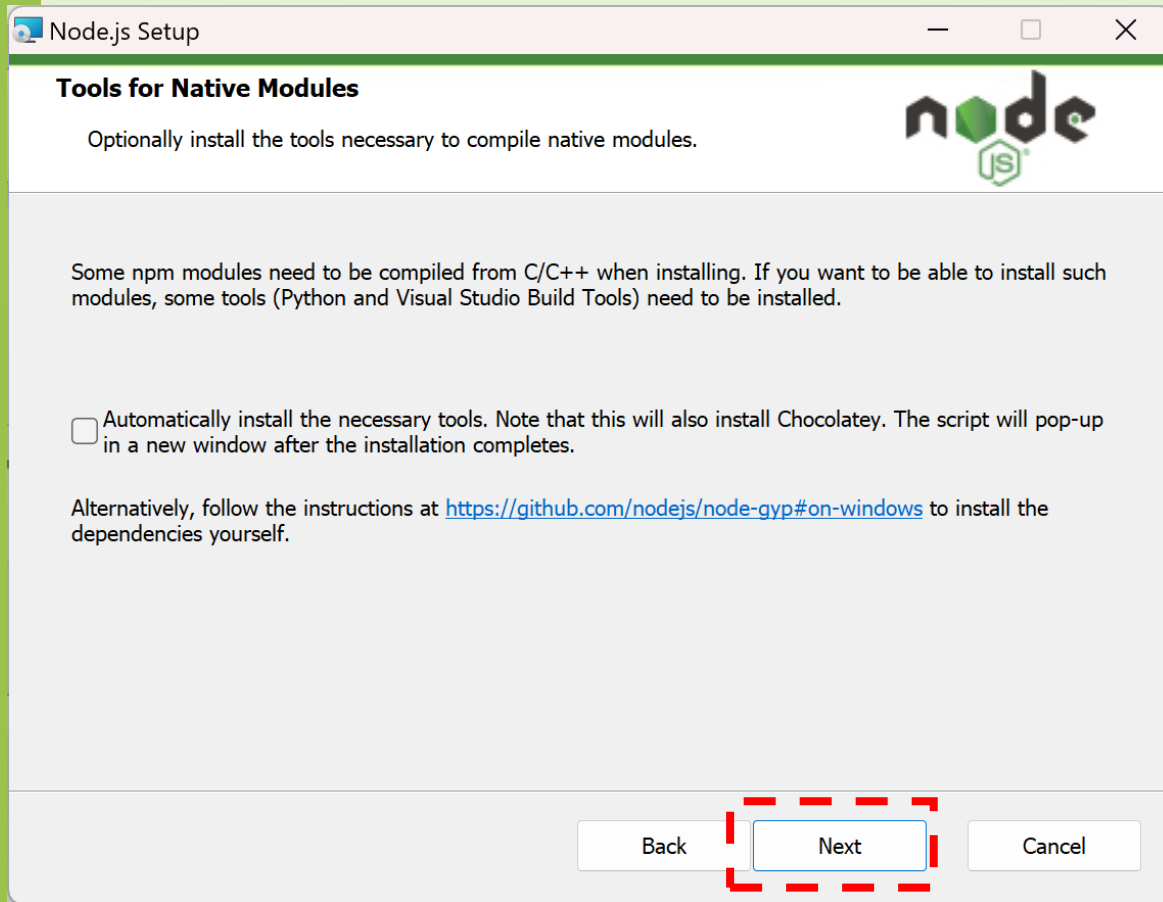


การติดตั้ง Node.js (3)

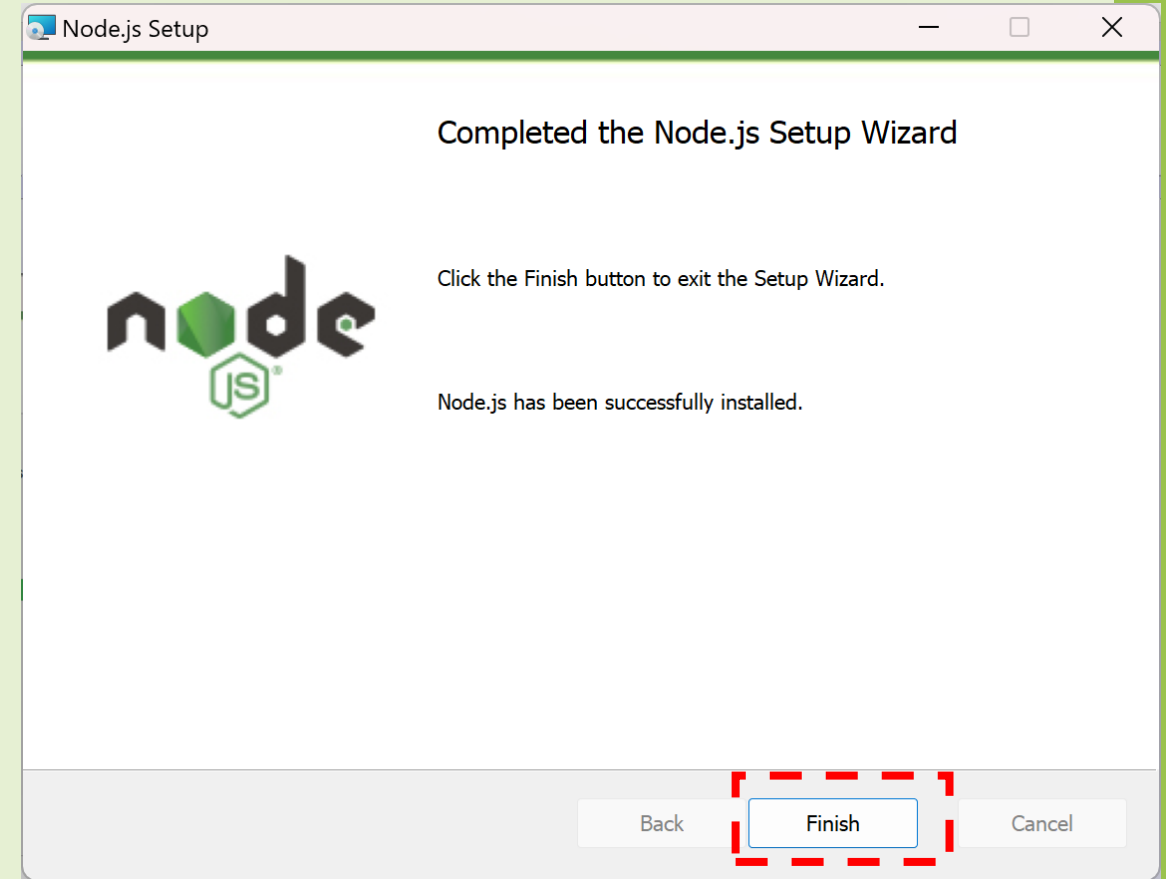
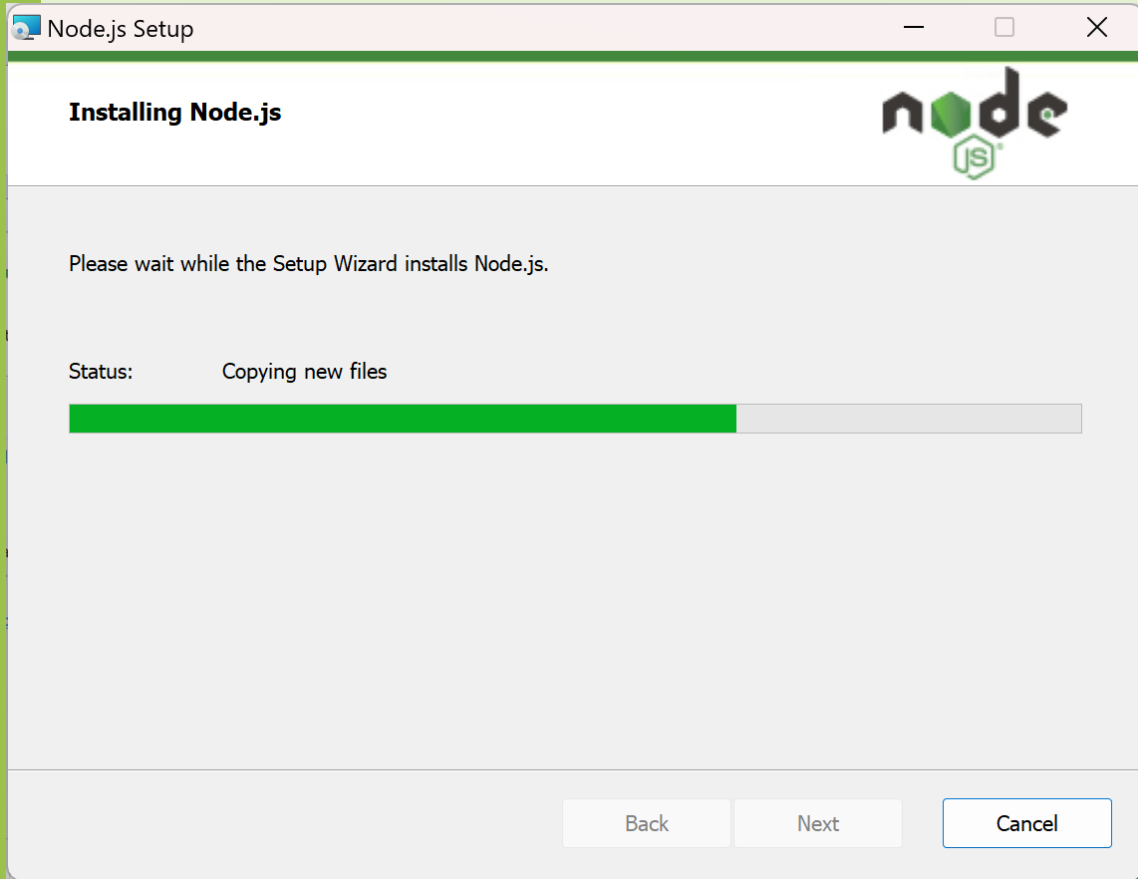
- เลือกตำแหน่งที่จะติดตั้ง



การติดตั้ง Node.js (4)



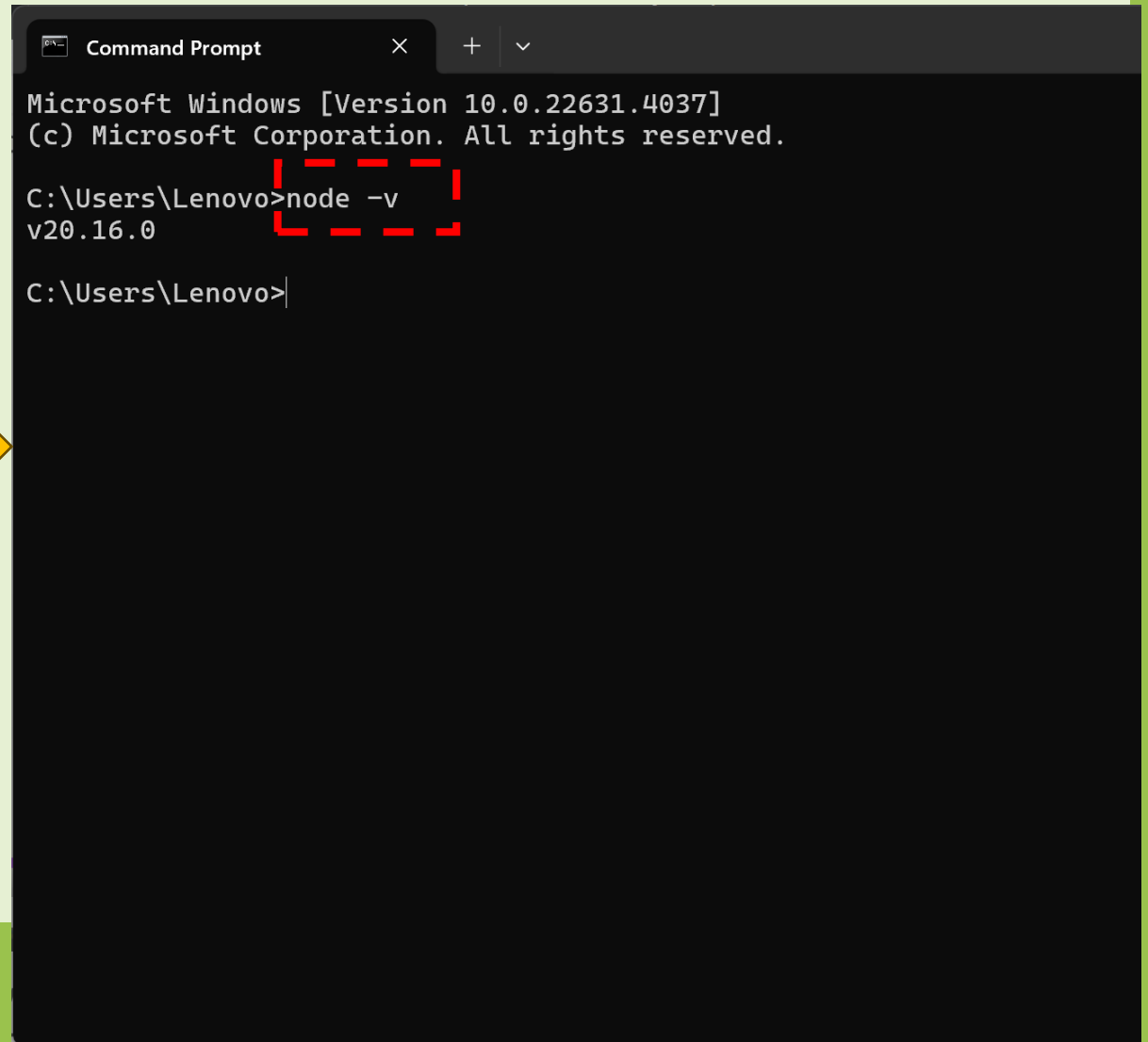
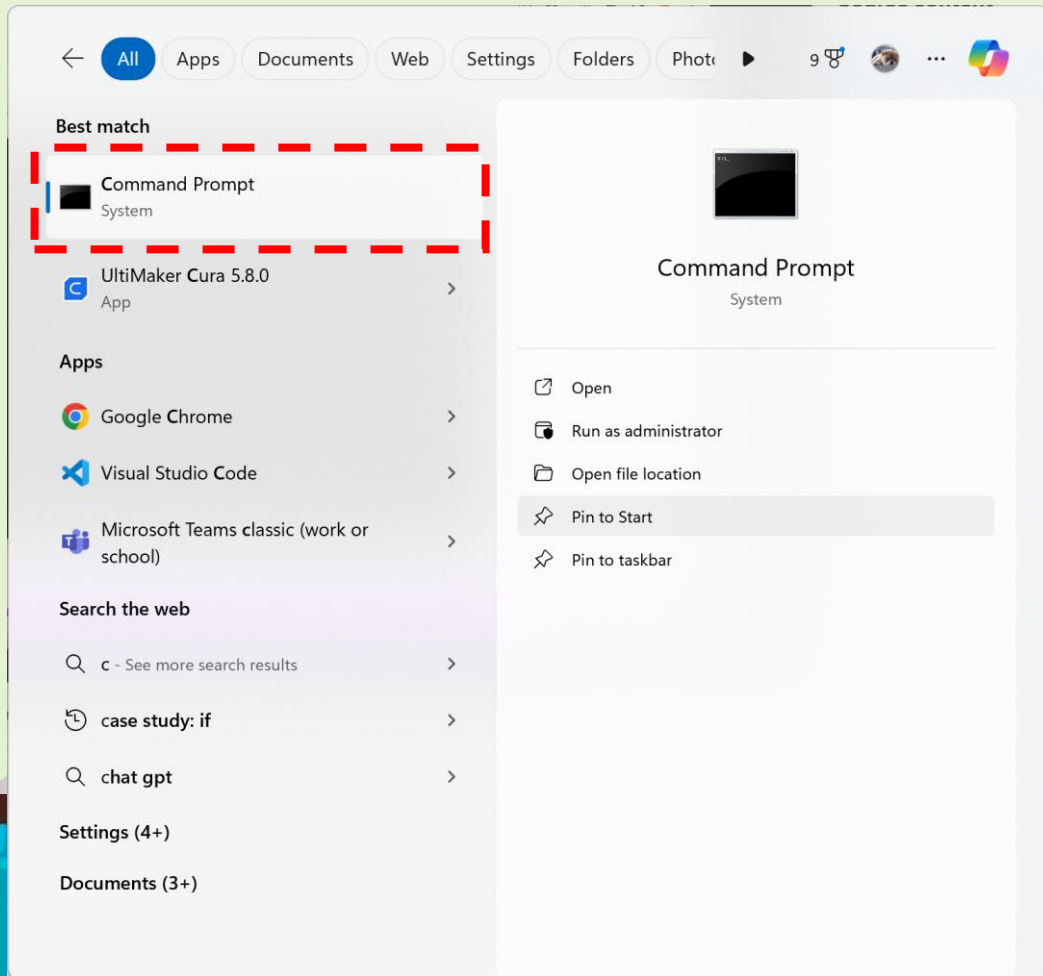
การติดตั้ง Node.js (5)



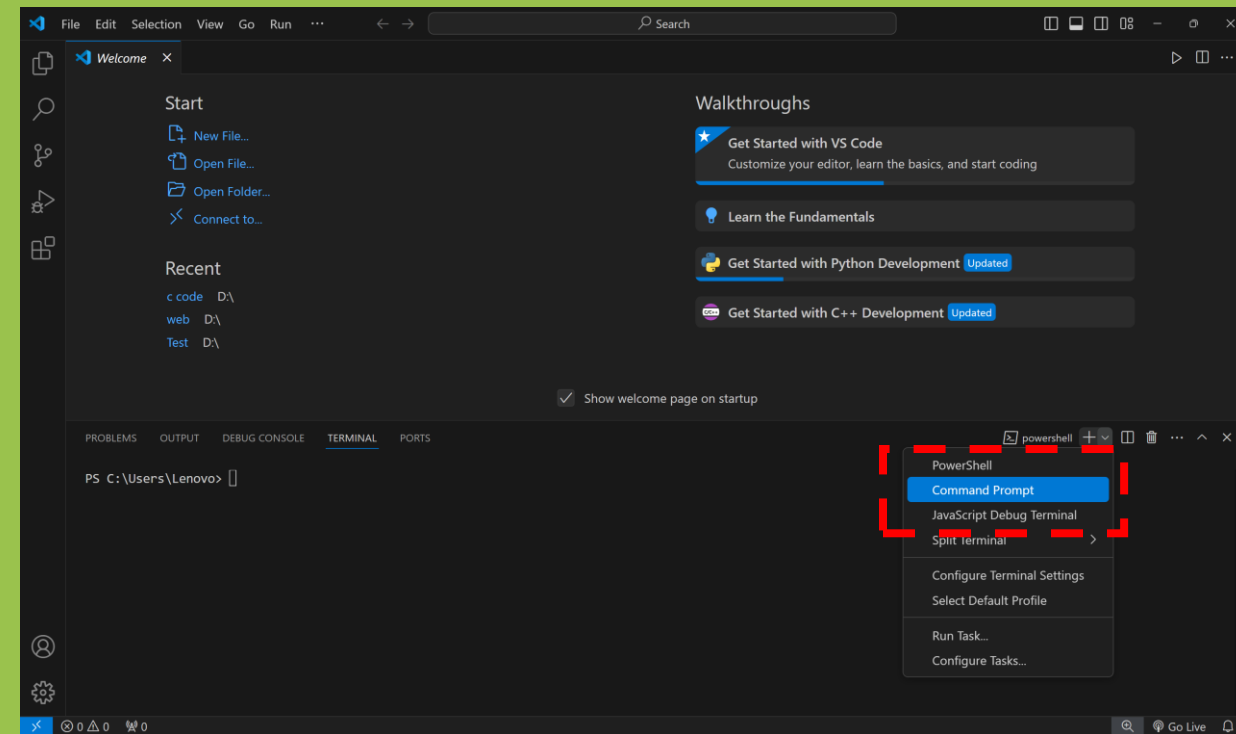
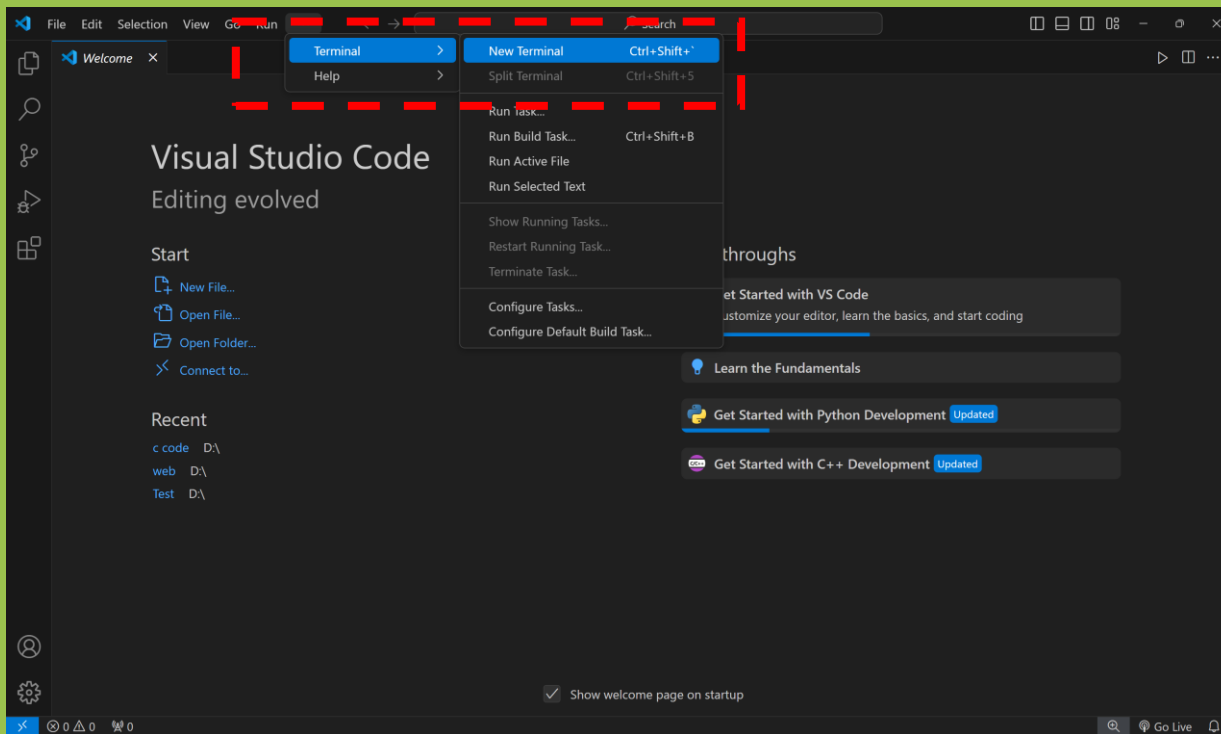
ทดลองใช้ Node.js หลังการติดตั้ง

8

- เปิด command Prompt พิมพ์ node -v
- จะมีการแสดงเลขเวอร์ชันของ Node.js ที่ติดตั้ง



การใช้งาน terminal ใน VS CODE



คลาส (Class)

- Class (คลาส) ใน JavaScript คือ แม่แบบ (blueprint) สำหรับสร้างวัตถุ (object) ที่มี พร็อพเพอร์ตี้ (property) และ เมธอด (method) แบบเดียวกัน
- Property (พร็อพเพอร์ตี้) คือ ตัวแปรที่เป็นของ object ที่สร้างจาก class
- Method คือ ฟังก์ชัน (function) ที่อยู่ภายใน class หรือ object ใช้สำหรับกำหนด พฤติกรรม (behavior) ของ object เช่น พูด, คำนวณ, แสดงผล สามารถเข้าถึง property ของ object ได้ด้วย this



รูปแบบการเขียน class

- ชื่อคลาสขึ้นต้นด้วยตัวพิมพ์ใหญ่ มีการตั้งชื่อคล้ายตัวแปร

```
class ClassName {  
    //สมาชิกในคลาส  
}
```

- เมื่อต้องการนำคลาสมาใช้งาน จะต้องสร้างตัวแปรอ้างอิงกับคลาส เรียกว่า (Instance) อินสแตนซ์
- การสร้างอินสแตนซ์ ใช้คีย์เวิร์ด new และตามด้วย ชื่อคลาส()

- เช่น

```
class Test {  
  
}  
  
const test1 = new Test()
```



Property (พร็อพเพอร์ตี้)

- Property (พร็อพเพอร์ตี้) คือ ตัวแปรที่เป็นของ object ที่สร้างจาก class
 - ถ้า Property มีค่าแน่นอน สามารถสร้างและกำหนดค่าได้โดยไม่ต้องใช้คีย์เวิร์ด const, let, var
 - ถ้า Property ต้องเปลี่ยนค่าตามการสร้าง Instance จะรับข้อมูลผ่าน Method ชื่อ constructor
 - 1 class สามารถมี constructor ได้แค่ 1 อัน
 - ภายใน constructor มีการระบุพารามิเตอร์ที่ใช้รับข้อมูลและนำไปกำหนดค่าเริ่มต้นให้ property โดยชื่อพารามิเตอร์สามารถเป็นชื่อเดียวกันกับ property ได้

```
class ClassName {  
    property1 = value  
    constructor(parameter1,parameter2){  
        this.property2 = parameter1  
        this.property3 = parameter2  
    }  
}
```

```
const instance1 = new ClassName(value1,value2)  
console.log(instance1.property1); // value  
console.log(instance1.property2); // value1  
console.log(instance1.property3); // value2
```



อินสแตนซ์ (Instance)

- ตัวแปรอ้างอิงกับคลาส เรียกว่า (Instance) อินสแตนซ์
- การสร้างอินสแตนซ์ ใช้คีย์เวิร์ด new และตามด้วย ชื่อคลาส() พร้อมการระบุค่า parameter ให้ครบ ยกเว้นข้อมูลที่มีค่าคงที่

```
class Member {  
    nationality = "Thai"  
    constructor(fname,lname,yearOfBirth){  
        this.Fname = fname  
        this.Lname = lname  
        this.age = new Date().getFullYear()-yearOfBirth  
    }  
}
```

```
const member1 = new Member("John", "Jones", 1990)  
console.log(member1.nationality) //Thai  
console.log(member1.Fname); //John  
console.log(member1.Lname); //Jones  
console.log(member1.age); //2025-1990=35
```



อินสแตนซ์ (Instance)

- ตัวแปรอ้างอิงกับคลาส เรียกว่า (Instance) อินสแตนซ์
- การสร้างอินสแตนซ์ ใช้คีย์เวิร์ด new และตามด้วย ชื่อคลาส() พร้อมการระบุค่า parameter ให้ครบ ยกเว้นข้อมูลที่มีค่าคงที่ หรือสามารถกำหนด parameter พร้อมค่าคงที่ได้

```
class Member {  
    constructor(fname,lname,yearOfBirth,nationality="Thai"){  
        this.Fname = fname  
        this.Lname = lname  
        this.age = new Date().getFullYear()-yearOfBirth  
        this.Nationality = nationality  
    }  
}
```

```
const member1 = new Member("John", "Jones", 1990)  
console.log(member1.Nationality) //Thai  
console.log(member1.Fname); //John  
console.log(member1.Lname); //Jones  
console.log(member1.age); //2025-1990=35
```



เมธอด (Method) ของ คลาส

- Method (เมธอด) คือ ฟังก์ชัน (function) ที่อยู่ภายในคลาส (class)
 - การสร้างเมธอด สามารถระบุชื่อได้เลยโดยไม่มีคำว่า function
 - ถ้ามีพารามิเตอร์ให้ระบุแบบเดียวกับการสร้างฟังก์ชัน
 - ถ้าต้องการอ้างถึง Property หรือ method ในคลาสเดียวกันสามารถใช้ this เช่น this.propertyName, this. MethodName()

```
class Person {
  nationality = "Thai";
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  introduce() {
    console.log(` สวัสดี ฉันชื่อ ${this.name} อายุ ${this.age} ปี สัญชาติ ${this.nationality}`);
  }
  haveBirthday() {
    this.age++; // ใช้ this เพื่อเข้าถึง property
    console.log(`${this.name} มีอายุเพิ่มขึ้นเป็น ${this.age} ปีแล้ว`);
  }
  celebrate() {
    console.log("ฉลองวันเกิด");
    this.haveBirthday(); // เรียกใช้ method ในคลาส
    this.introduce();    // เรียก method อื่นต่อ
  }
}

const p1 = new Person("สมชาย", 25);

// เรียกใช้เมธอด
p1.introduce(); // สวัสดี ฉันชื่อ สมชาย อายุ 25 ปี สัญชาติ Thai
p1.haveBirthday(); // สมชาย มีอายุเพิ่มขึ้นเป็น 26 ปีแล้ว
p1.celebrate(); // ฉลองวันเกิด
// สมชาย มีอายุเพิ่มขึ้นเป็น 27 ปีแล้ว
// สวัสดี ฉันชื่อ สมชาย อายุ 27 ปี สัญชาติ Thai
```



เมธอดแบบ static

- Static Method จะถูกผูกกับ คลาส (class) ไม่ได้ผูกกับ อินสแตนซ์ (object)
- เรียกใช้ได้โดยตรงจาก ชื่อคลาส โดยไม่ต้อง new
- ใช้ static ระบุหน้า method เพื่อสร้าง Static Method
 - Static → เรียก Static ได้โดยตรง
 - Static → เรียก Instance ไม่ได้ (ต้องสร้าง object ก่อน)
 - Instance → เรียก Static ได้ถ้าอ้างอิงผ่าน ClassName.method()

```
class Example {  
  sayHello() {  
    console.log("Hello from instance method");  
  }  
  static greet() {  
    console.log("Hello from static method");  
  }  
  static callAnotherStatic() {  
    this.greet();    // static เรียก static ขึ้นได้  
  }  
  static callInstance() {  
    // this.sayHello(); // Error:  
    // ต้องสร้าง object ก่อน  
    const obj = new Example();  
    obj.sayHello();  
  }  
}
```

```
Example.callAnotherStatic(); // Hello from static method  
Example.callInstance();     // Hello from instance method
```



การสืบทอดระหว่างคลาส (Class Inheritance)

- Inheritance (การสืบทอด) คือความสามารถที่คลาสหนึ่ง (เรียกว่า คลาสลูก / subclass) สามารถ สืบทอดคุณสมบัติ (property) และ พฤติกรรม (method) จากอีกคลาสหนึ่ง (เรียกว่า คลาสแม่ / superclass) ได้
- ทำให้ไม่ต้องเขียนโค้ดซ้ำ สามารถนำมาใช้ซ้ำและขยายความสามารถเพิ่มเติม ได้

```
class Subclass extends Superclass{  
}
```

- extends → ใช้ให้คลาสลูกสืบทอดจากคลาสแม่

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  introduce() {  
    console.log(`ฉันชื่อ ${this.name}`);  
  }  
}
```

```
class Student extends Person {  
}
```

```
const s1 = new Student("สมหญิง");  
s1.introduce(); // ฉันชื่อ สมหญิง
```



การโอเวอร์ไรด์ (Override) เมธอดของ Superclass

- ภายใน Subclass สามารถมีเมธอดได้
- สามารถเพิ่มการทำงานที่มีอยู่เดิมใน Superclass ได้ เรียกว่า การโอเวอร์ไรด์ (Override) โดยมีข้อกำหนดดังนี้
 - ชื่อเมธอดและรูปแบบการเขียนชื่อตรงกับ Superclass
 - รูปแบบการรับพารามิเตอร์ต้องตรงกัน
- คีย์เวิร์ด `super()`
 - ใช้ใน constructor เพื่อเรียก constructor ของ คลาสแม่
 - ใช้ใน method เพื่อเรียก method เดิมของคลาสแม่

```
class Person {
  constructor(name) {
    this.name = name;
  }
  introduce() {
    console.log(`ฉันชื่อ ${this.name}`);
  }
}
```

```
class Student extends Person {
  constructor(name, studentId) {
    super(name);
    this.studentId = studentId;
  }
}
```

```
// override introduce แต่ยังเรียก method เดิมของแม่ด้วย
introduce() {
  super.introduce(); // เรียก introduce ของ Person
  console.log(`รหัสนักศึกษา: ${this.studentId}`);
}
```

```
const s1 = new Student("สมหญิง", "S12345");
s1.introduce(); // ฉันชื่อ สมหญิง
// รหัสนักศึกษา: S12345
```



การสร้างและส่งออกโมดูล

- โมดูล = ไฟล์โค้ดที่ แยกออกมา เพื่อใช้ซ้ำหรือนำไปใช้ในไฟล์อื่น
- ช่วยให้โค้ดอ่านง่าย, แบ่งส่วนการทำงานชัดเจน และ จัดการ scope ได้ดีขึ้น
 - เมื่อใช้ export และ import ตัวแปร/ฟังก์ชันในแต่ละไฟล์จะถูก จำกัด scope อยู่ในโมดูลของมันเอง
 - จะมองไม่เห็นจากไฟล์อื่น เว้นแต่จะ export ออกมา และ import เข้าไป
- JavaScript มี 2 แบบหลัก:
 - CommonJS (ใช้ใน Node.js) → ใช้ require() และ module.exports
 - ES Module (สมัยใหม่) → ใช้ import และ export
- ถ้าใช้ Node.js รุ่นใหม่ ($\geq v14$) ต้องใส่ `"type": "module"` ใน package.json ถึงจะใช้ import/export ได้
- ถ้าไม่ใส่ จะต้องใช้ CommonJS (require/module.exports)



การสร้างและส่งออกโมดูล แบบ ES Module

- ต้องใส่ `"type": "module"` ใน `package.json` ถึงจะใช้ `import/export` ได้
- ใช้ `export` ตามด้วย `{}` ภายในส่งสิ่งที่ต้องการออกไป
- ใช้ `export default` ในการส่งออกค่าเริ่มต้นของไฟล์ (1 ไฟล์มีได้แค่ 1) เวลา `import` ไม่ต้องใส่ `{}` และตั้งชื่ออะไรก็ได้

ไฟล์ `math.js`

```
export const Pi = 3.141
export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}

// ส่งออกแบบ default export (มีได้ 1 อันต่อไฟล์)
export default function subtract(a, b) {
  return a - b;
}
```

หรือ

ไฟล์ `math.js`

```
const Pi = 3.141
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

function subtract(a, b) {
  return a - b;
}

export {Pi, add, multiply}
export default subtract
```

ใส่ `export` หน้าตัวแปรหรือฟังก์ชันที่ต้องการส่งออก

สร้างตัวแปรหรือฟังก์ชัน แล้วใช้ `export` ส่งออก

นำเข้าโมดูล

- ใช้ import ตามด้วย {} ภายในใส่ชื่อตัวแปรหรือฟังก์ชันที่ต้องการนำเข้ามาใช้งาน
- ใช้ import ตามด้วย ชื่อตัวแปรหรือฟังก์ชัน default ที่ต้องการนำเข้ามาใช้งาน
- ใช้ from ตามด้วยตำแหน่งของโมดูล
- ใช้ ./ แทนไฟล์ที่ต้องการใช้โมดูลอยู่ในไดเรกทอรีเดียวกัน

ไฟล์ app.js

```
// import แบบ named
import {Pi, add, multiply } from './math.js';

// import แบบ default
import subtract from './math.js';

// หรือใช้ import subtract, { Pi, add, multiply } from './math.js';

console.log(add(5, 3)) // 8
console.log(multiply(4, 2)) // 8
console.log(subtract(10, 7)) // 3
console.log(Pi) //3.141
```

สามารถใช้ node ชื่อไฟล์.js ในการรัน

```
const Pi = 3.141
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

function subtract(a, b) {
  return a - b;
}

export {Pi, add, multiply }
export default subtract
```

ไฟล์ math.js

```
// import แบบ named
import {Pi, add, multiply } from './math.js';

// import แบบ default
import subtract from './math.js';

console.log(add(5, 3))           // 8
console.log(multiply(4, 2))      // 8
console.log(subtract(10, 7))     // 3
console.log(Pi)                  // 3.141
```

ไฟล์ app.js

นำเข้าและใช้โมดูล

ทดสอบการใช้โมดูล ดัสนการรัน

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Lenovo\Desktop\test> node app.js
8
8
3
3.141
```

การสร้างและส่งออกโมดูล



การใช้งานโมดูลในเพจ (.html)

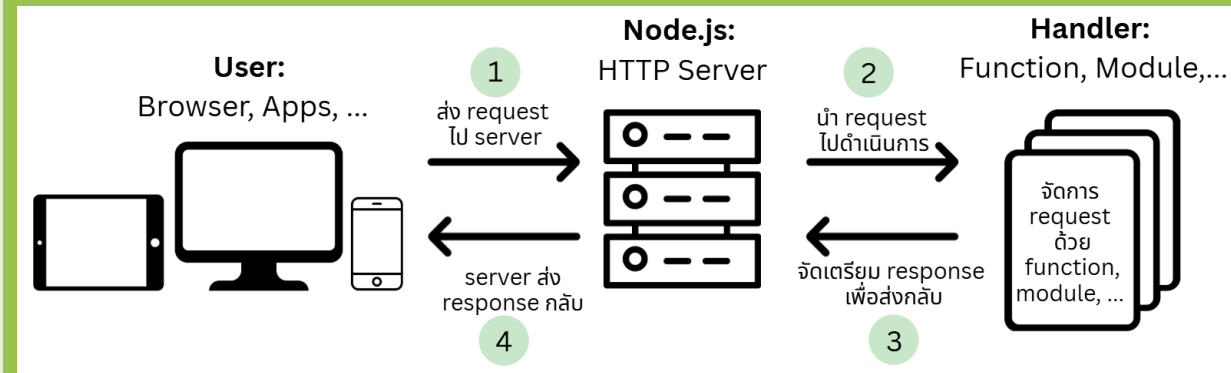
- ใช้โมดูลภายใน `<script type="module"> </script>`
- โมดูลใช้ภายในแท็ก `<script>` ที่เรียกใช้เท่านั้น ไม่สามารถใช้ข้ามแท็ก `<script>` ได้
- สามารถตั้งชื่อโมดูลด้วยชื่อใหม่ได้ด้วยการใช้ `as`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <p id="idCard"></p>
  <script type="module">
    import {add} from './math.js';
    let p = document.getElementById('idCard')
    p.innerText = add(5,20)
  </script>
</body>
</html>
```

Node.js กับการทำงานฝั่ง Server



- **Node.js** คือ JavaScript runtime (สภาพแวดล้อมสำหรับรันโค้ด JavaScript บนฝั่งเซิร์ฟเวอร์) ที่ทำงานอยู่นอกเว็บเบราว์เซอร์ โดยสร้างขึ้นบน V8 Engine (ตัวเดียวกับที่ Chrome ใช้ประมวลผล JavaScript)
- **V8 Engine** คือ JavaScript Engine ที่พัฒนาโดย Google ทำหน้าที่คือ แปลง (compile) โค้ด JavaScript → เป็น Machine Code ที่ CPU เข้าใจได้โดยตรง
- **Node.js** ถูกสร้างขึ้นมาเพื่อการทำงานในรูปแบบของ Web server
- **Web Server** เป็น ซอฟต์แวร์หรือฮาร์ดแวร์ที่ทำหน้าที่ให้บริการข้อมูลเว็บ (รับคำขอ (Request) จากผู้ใช้งาน เว็บเบราว์เซอร์ (Client) และ ส่งข้อมูลตอบกลับ (Response) เช่น เว็บเพจ (HTML, CSS, JS), ไฟล์ภาพ, หรือข้อมูล API กลับไปให้ผู้ใช้งาน)
- **HTTP Server** เป็นส่วนหนึ่งของ Web Server ที่สื่อสารกับ Client ผ่าน โปรโตคอล HTTP/HTTPS
- **HTTP Server** ซอฟต์แวร์/กระบวนการที่รับ-ส่งข้อมูลระหว่าง Client (เช่น Browser) และ Server โดยใช้ HTTP (HyperText Transfer Protocol) เป็นโปรโตคอลหลักทำงานบน **พอร์ตมาตรฐาน 80 (HTTP) หรือ 443 (HTTPS)** ทำหน้าที่:
 - รับ Request (คำขอ) จาก Client
 - ประมวลผล (เช่น อ่านไฟล์, รันโค้ด, ติดต่อ Database)
 - ส่ง Response (คำตอบ) กลับไป เช่น HTML, JSON, รูปภาพ



npm

- npm คือ package manager command ที่ใช้ในการ ติดตั้ง อัปเดต ลบ หรือจัดการแพ็คเกจหรือโมดูลต่างๆ ที่เกี่ยวข้องกับ Node.js
- แสดงเวอร์ชันของ npm
`npm -v`
- ใช้ npm เพื่อติดตั้งแพ็คเกจ

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Lenovo\Desktop\test> npm -v
10.8.1
PS C:\Users\Lenovo\Desktop\test> 
```



การสร้างองค์ประกอบพื้นฐานของ Node Application

- Node Application คือ โปรแกรมหรือแอปพลิเคชันที่พัฒนาด้วย Node.js โดยใช้ JavaScript (หรือ TypeScript) เพื่อรันบนฝั่งเซิร์ฟเวอร์ (Server-side)
- การสร้างองค์ประกอบพื้นฐานของ Node Application
 1. เปิด vs code เลือกโฟลเดอร์ที่ต้องการเก็บโปรเจคหรือแอป
 2. สร้างไฟล์ package.json ด้วยคำสั่ง `npm init -y`
 3. สร้างไฟล์หลักของแอป คือ index.js โดยต้องแก้ไขที่พรีอเบเพอร์ตี้ main ที่ไฟล์ package.json ให้ตรงกัน
 4. ใส่ `"type": "module"` ใน package.json ถึงจะใช้ import/export ได้

```
✓ test
  JS index.js
  {} package.json
```

"scripts" = เก็บคำสั่งลัดที่รันผ่าน npm run



ไฟล์ package.json

- "name" = เก็บชื่อโปรเจกต์หรือแพ็คเกจ
- "version" = เก็บ เวอร์ชันของโปรเจกต์ มีรูปแบบ MAJOR.MINOR.PATCH
 - MAJOR → เปลี่ยนแปลงครั้งใหญ่, ไม่เข้ากันกับเวอร์ชันเก่า (breaking change)
 - MINOR → เพิ่มฟีเจอร์ใหม่ แต่ยังเข้ากันได้กับของเดิมได้
 - PATCH → แก้บั๊กเล็ก ๆ
 - เช่น "version": "2.3.4" → major 2, minor 3, patch 4
- "scripts" = เก็บคำสั่งที่รันผ่าน npm run เช่น npm run test
- "keywords" = array ของคำค้น สำหรับ package ไม่ใช่ dependencies ต้องตั้งค่าเอง (ใส่หรือไม่ใส่ก็ได้)
- "author" = ระบุเจ้าของหรือทีมผู้พัฒนา
- "license" = บอกว่าซอฟต์แวร์นี้แจกภายใต้สัญญาอนุญาตแบบไหน "ISC" เป็นค่า default ของ npm init
- "description" = คำอธิบายสั้น ๆ ของโปรเจกต์

{} package.json X

test > {} package.json > ...

```

1  {
2    "name": "test",
3    "version": "1.0.0",
4    "main": "index.js",
5    "type": "module",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "description": ""
13 }
```

การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js

- หลังจากสร้างองค์ประกอบของ Node Application จะดำเนินการให้การทำงานเป็นแบบ web server โดยกำหนดโค้ดที่ไฟล์หลักของแอป (index.js) ดังนี้
- 1. โมดูลที่จำเป็นต้องใช้ http เพื่อควบคุมการทำงานของ Server
 - รับ-ส่งข้อมูล
 - กำหนดเส้นทาง (Routing) เพื่อใช้ตรวจสอบ req.url และ req.method (GET, POST ฯลฯ) ตอบกลับตาม path ที่ร้องขอ
 - สื่อสารกับ API อื่น ๆ โดย http module ใช้เป็น HTTP Client ได้ด้วย (ยิง request ไปยัง server อื่น)

```
// นำเข้าโมดูล http
import http from "http";
```

การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js

- หลังจากสร้างองค์ประกอบของ Node Application จะดำเนินการให้การทำงานเป็นแบบ web server โดยกำหนดโค้ดที่ไฟล์หลักของแอป (index.js) ดังนี้
- 2. ใช้ `http.createServer((req, res) => {...})` เพื่อสร้าง HTTP Server กำหนด callback function สำหรับจัดการ request/response
 - req, request = ข้อมูล request จาก client (url, method, headers)
 - res, response = object สำหรับตอบกลับ client

```
// นำเข้าโมดูล http
import http from "http";

// สร้าง server
const app = http.createServer((req, res) => {

})
```

การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js

- หลังจากสร้างองค์ประกอบของ Node Application จะดำเนินการให้การทำงานเป็นแบบ web server โดยกำหนดโค้ดที่ไฟล์หลักของแอป (index.js) ดังนี้
- 3. res.statusCode() ใช้กำหนด HTTP status

```
// นำเข้าโมดูล http
import http from "http";

// สร้าง server
const app = http.createServer((req, res) => {
  res.statusCode = 200;
})
```

การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js

- หลังจากสร้างองค์ประกอบของ Node Application จะดำเนินการให้การทำงานเป็นแบบ web server โดยกำหนดโค้ดที่ไฟล์หลักของแอป (index.js) ดังนี้
- 4. res.setHeader () ใช้สำหรับ ตั้งค่า HTTP header ก่อนที่จะส่ง Response กลับไปยัง Client (บอกข้อมูลเพิ่มเติมให้กับ client เกี่ยวกับ response ที่จะส่งไป เช่น ชนิดข้อมูล, การเข้ารหัส, การควบคุม cache ฯลฯ)
 - Content-Type เป็น HTTP Header ที่บอกให้ Client (เช่น Browser, Postman) รู้ว่า ข้อมูลที่ Server ส่งกลับมาเป็นชนิดอะไร เพื่อให้ Client จะได้ตีความและแสดงผลได้ถูกต้อง
 - "text/plain; charset=utf-8" → ข้อมูลเป็นข้อความธรรมดา (Plain Text) ไม่ใช่ HTML, JSON, XML หรือไฟล์ประเภทอื่น กำหนดการเข้ารหัสอักขระ (character encoding) UTF-8 = มาตรฐานการเข้ารหัสตัวอักษรที่รองรับภาษาเกือบทุกภาษาในโลก รวมถึง ภาษาไทยถ้าไม่ใส่ อาจทำให้ตัวอักษรพิเศษหรือภาษาไทยแสดงผลเป็น "ภาษาต่างดาว"
 - "text/html; charset=utf-8" → ข้อมูลเป็น HTML
 - "application/json; charset=utf-8" → ข้อมูลเป็น JSON
 - "text/css" → ข้อมูลเป็น CSS
 - "application/javascript" → ข้อมูลเป็น JS
 - "image/png" → ข้อมูลเป็นรูปภาพ PNG

```
// นำเข้าโมดูล http
import http from "http";

// สร้าง server
const app = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain; charset=utf-8");
})
```



การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js

- หลังจากสร้างองค์ประกอบของ Node Application จะดำเนินการให้การทำงานเป็นแบบ web server โดยกำหนดโค้ดที่ไฟล์หลักของแอป (index.js) ดังนี้
- 5. res.end() ใช้ส่งข้อความตอบกลับแล้วปิดการเชื่อมต่อ

```
// นำเข้าโมดูล http
import http from "http";

// สร้าง server
const app = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain; charset=utf-8");
  res.end("Hello World");
})
```



การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js

- หลังจากสร้างองค์ประกอบของ Node Application จะดำเนินการให้การทำงานเป็นแบบ web server โดยกำหนดโค้ดที่ไฟล์หลักของแอป (index.js) ดังนี้
- 6. `const PORT = 3000;` กำหนดหมายเลขพอร์ต (Port) ที่เซิร์ฟเวอร์จะเปิดรอรับการเชื่อมต่อพอร์ตคือ “ช่องทาง” ในเครื่องคอมพิวเตอร์ที่ใช้สื่อสารกัน เช่น
 - 80 → HTTP
 - 443 → HTTPS
 - ในที่นี้เราตั้งเป็น 3000 (นิยมใช้เลข 3000, 4000, 5000 สำหรับ dev/test)
- 7. `app.listen(PORT, ...)` เพื่อเปิดรอรับการเชื่อมต่อจาก browser/client โดยสั่งให้เซิร์ฟเวอร์เริ่มฟัง (listen) คำขอจาก Client ที่พอร์ตที่กำหนด ฟังก์ชันนี้มาจาก object ที่ได้จาก `http.createServer()` เมื่อ server พร้อมทำงานในคำสั่งที่อยู่ภายใน `app.listen()`

```
// นำเข้าโมดูล http
import http from "http";

// สร้าง server
const app = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain; charset=utf-8");
  res.end("Hello World");
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```



การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js

- สามารถใช้งาน Web Server ของได้ด้วยการใช้คำสั่ง node <ชื่อไฟล์หลักของแอป>
- เช่น
- `node index.js`
- หลังจากที่เรารันโปรแกรมแล้ว Cursor บน Command line จะค้างอยู่แบบนั้นตลอดไป นั่นเป็นเพราะว่าเว็บเซิร์ฟเวอร์กำลังรอรับคำร้องขออยู่แบบ Asynchronous และมันจะทำงานตลอดไปจนกว่าจะมีการกดปุ่ม **CTRL+C** เพื่อออกจากโปรแกรม
- ใส่ URL ใน Web Browser
- `http://localhost:3000/`
(ใช้ตามที่กำหนด port)
- `http://localhost:3000`
 - localhost เป็น hostname พิเศษที่ชี้ไปยัง เครื่องของเราเองโดยค่าเริ่มต้นระบบปฏิบัติการจะแปลง localhost → 127.0.0.1 (IPv4) หรือ ::1 (IPv6) ผ่านไฟล์ hosts
 - เวลาเรา `app.listen(3000)` → เข้าได้ทั้ง `http://localhost:3000` และ `http://127.0.0.1:3000`

```
// นำเข้าโมดูล http
import http from "http";

// สร้าง server
const app = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain; charset=utf-8");
  res.end("Hello World");
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Lenovo\Desktop\test\test> node index.js
Server running at http://localhost:3000
```



การตรวจสอบ Status Code

สามารถตรวจสอบความหมายของ Status Code ได้โดยพิมพ์

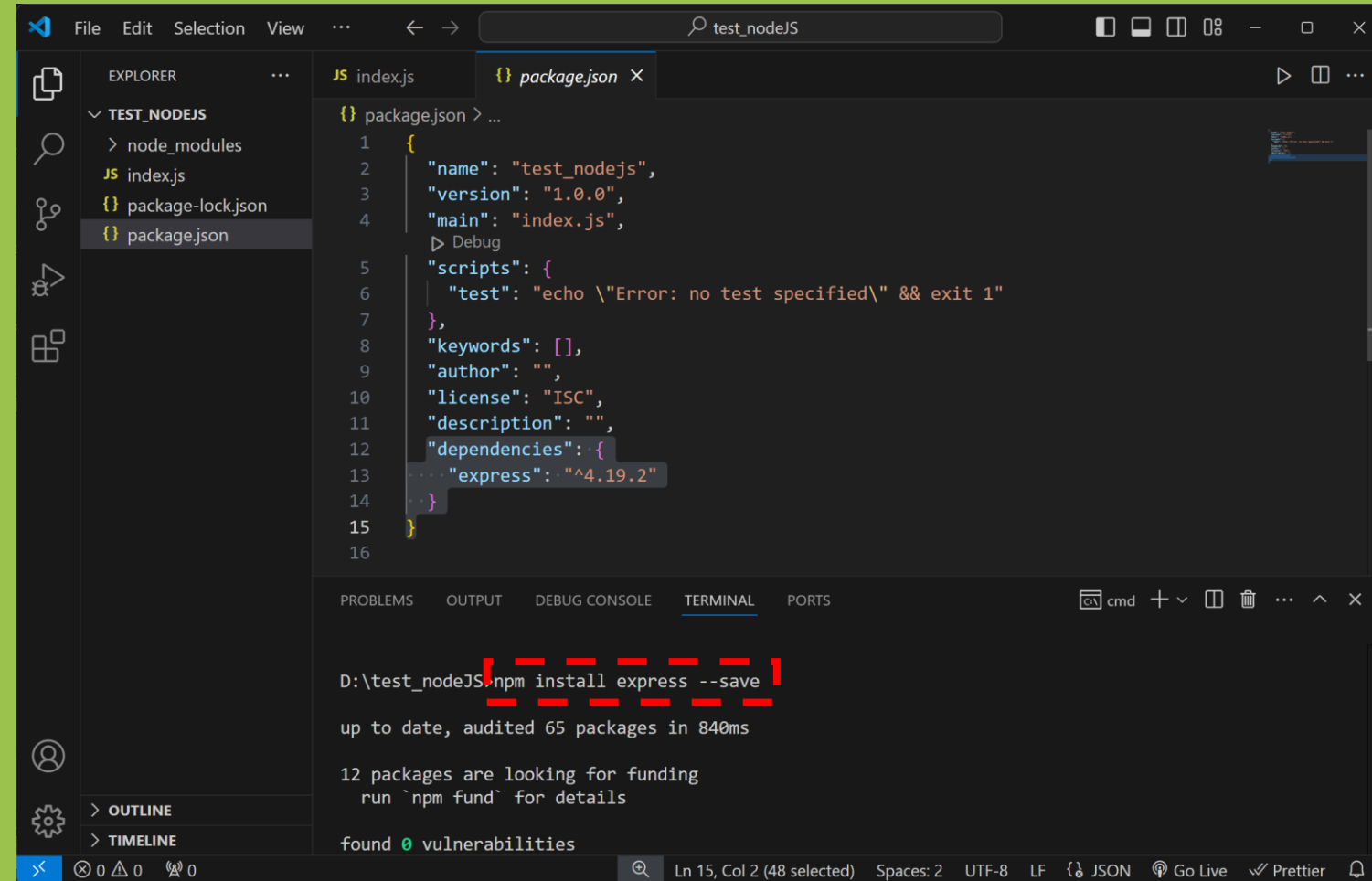
```
node --print "http.STATUS_CODES"
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Lenovo\Desktop\test\test> node --print "http.STATUS_CODES"
{
  '100': 'Continue',
  '101': 'Switching Protocols',
  '102': 'Processing',
  '103': 'Early Hints',
  '200': 'OK',
  '201': 'Created',
  '202': 'Accepted',
  '203': 'Non-Authoritative Information',
  '204': 'No Content',
  '205': 'Reset Content',
  '206': 'Partial Content',
  '207': 'Multi-Status',
  '208': 'Already Reported',
  '226': 'IM Used',
  '300': 'Multiple Choices',
  '301': 'Moved Permanently',
```

ติดตั้ง Express.js

- ใช้เพื่อทำให้การพัฒนาเว็บแอปพลิเคชันบน Node.js ทำได้ง่ายขึ้น และ Express.js มี Express.js จะช่วยทำให้จัดการกับระบบหลังบ้าน เช่น การจัดการ Routing ที่ง่าย, ฟังก์ชันช่วยสำหรับ HTTP
- โดยที่ Express.js เป็น Framework ตัวหนึ่งของ NodeJS ที่จะช่วยจัดการเรื่องของ APIs ได้อย่างดี
- ติดตั้ง Express.js ภายในโปรเจกต์ผ่านคำสั่ง
`npm install express`
- หลังติดตั้งเสร็จ จะมีโฟลเดอร์ `node_modules/` และไฟล์ `package-lock.json` เพิ่มขึ้นและใน `package.json` จะมี "express" เพิ่มใน dependencies ในไฟล์ `package.json`



```
{} package.json > ...
1  {
2    "name": "test_nodejs",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": "",
12   "dependencies": {
13     "express": "^4.19.2"
14   }
15 }
16
```

```
D:\test_nodeJS> npm install express --save
up to date, audited 65 packages in 840ms
12 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

การใช้ Express.js ในการสร้าง Web Server

ไม่ใช้ Express.js

```
// นำเข้าโมดูล http
import http from "http";

// สร้าง server
const app = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain; charset=utf-8");
  res.end("Hello World");
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

ใช้ Express.js

```
// นำเข้า express แบบ ES Module
import express from "express";

// สร้าง server ด้วย express()
const app = express();

// สร้าง route (เมื่อ user เปิดหน้า / จะตอบกลับด้วยข้อความ)
app.get('/', (req, res) => {
  // res.status(200)
  // res.type('text/plain')
  // res.send("Hello world")

  // หรือ res.status(200).type('text/plain').send("Hello world");
});

// กำหนดพอร์ต
const PORT = 3000;

// สั่งให้ server เริ่มฟังที่พอร์ต
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```



nodemon

- จากการเขียนโปรแกรมที่ผ่านมาเมื่อทำการรันโปรแกรม nodejs นั้นต้องทำการรันคำสั่ง node index.js (ไฟล์เริ่มแรก que เมื่อ run จะเป็น server) ของทุกครั้งเวลาเมื่อแก้ไขโปรแกรม และทำการ save โปรแกรมที่เขียนขึ้น
- Nodemon คือเครื่องมือที่ช่วย Restart node server ให้โดยอัตโนมัติเมื่อมีการ แก้ไขโปรแกรม
- Nodemon เป็น Open-source ที่สามารถนำมาใช้งานได้ฟรีโดยไม่มีค่าใช้จ่าย
- ขั้นตอนการใช้งาน Nodemon
 - ติดตั้ง Nodemon โดยใช้คำสั่ง `npm install --save-dev nodemon`
 - Run nodemon โดยใช้ `npx nodemon index.js` หลังจากนั้น Node.js จะถูก restart ทันทีที่มีการ Save file



nodemon

- จากการเขียนโปรแกรมที่ผ่านมาเมื่อทำการรันโปรแกรม nodejs นั้นต้องทำการรันคำสั่ง node index.js (ไฟล์เริ่มแรก que เมื่อ run จะเป็น server) ของทุกครั้งเวลาเมื่อแก้ไขโปรแกรม และทำการ save โปรแกรมที่เขียนขึ้น
- Nodemon คือเครื่องมือที่ช่วย Restart node server ให้โดยอัตโนมัติเมื่อมีการ แก้ไขโปรแกรม
- Nodemon เป็น Open-source ที่สามารถนำมาใช้งานได้ฟรีโดยไม่มีค่าใช้จ่าย
- ขั้นตอนการใช้งาน Nodemon
 - ติดตั้ง Nodemon โดยใช้คำสั่ง `npm install --save-dev nodemon`
 - Run nodemon โดยใช้ `npx nodemon index.js` หลังจากนั้น Node.js จะถูก restart ทันทีที่มีการ Save file



การกำหนดเส้นทางพื้นฐาน

- ใช้ `app.use()` ในการกำหนดเส้นทางพื้นฐาน จะไม่ขึ้นกับ path ใด
- เช่น

```
app.use((req,res)=>{  
  response.send('Hello word')  
})
```

- จะกำหนด URL เป็นอะไรก็ได้ทำงาน
 - `http://localhost:3000`
 - `http://localhost:3000/test`
 - `http://localhost:3000/hello/test`
- สามารถกำหนด path เริ่มต้นได้ เช่น

```
app.use('/about', (req,res)=>{  
  res.send('Hello word')  
})
```

- จะกำหนด URL ที่เริ่มต้นด้วย `/about`
 - `http://localhost:3000/about`
 - `http://localhost:3000/about /test`
 - `http://localhost:3000/about /hello/test`



การกำหนดเส้นทางด้วยการใช้เมธอด get

- เส้นทางหรือรูปแบบ URL ในแอปพลิเคชันเว็บที่ถูกกำหนดให้เชื่อมโยงกับฟังก์ชันหรือโค้ดเฉพาะ เมื่อผู้ใช้ส่งคำขอไปยังแอปพลิเคชัน (เช่น การเข้าชม URL)
- เมธอด get ใช้สำหรับขอข้อมูลจากเซิร์ฟเวอร์ โดยไม่มีการเปลี่ยนแปลงข้อมูลที่อยู่บนเซิร์ฟเวอร์ เช่น การเรียกดูเพจเว็บหน้าแรก การขอข้อมูลรายการสินค้า เป็นต้น
- รูปแบบการกำหนด :

```
app.get(PATH, HANDLER)
```

- โดย
 - **app** คือ instance ของ express (เข้าใจง่ายๆ "ฉันทัน (app)" คือตัวแทนของ express)
 - **METHOD** คือ HTTP request method ใช้เป็นตัวเล็ก หมายถึง ทำการเรียกข้อมูลผ่าน HTTP ด้วยวิธี * get | post | put | delete
 - **PATH** คือ URL หรือ ส่วนของโดเมนนับตั้งแต่ "/" ตัวที่ 3 เป็นต้นไป เช่น *http://niik.in/forum จะได้ PATH คือ "/fourn" ถ้าเป็น Root PATH ก็คือ "/"
 - **HANDLER** คือฟังก์ชันที่กำหนดให้ทำงาน เมื่อ PATH ตรงกับค่าที่กำหนด

การกำหนดเส้นทางด้วยการใช้เมธอด get

- จะทำงานเมื่อ GET Request ที่ path / หรือ http://localhost:3000/

```
app.get('/', (req, res) => {  
  res.status(200)  
  res.type('text/plain')  
  res.send('Hello world')  
});
```

- จะทำงานเมื่อ GET Request ที่ path /test หรือ http://localhost:3000/test

- ```
app.get('/test', (req, res) => {
 res.status(200).type('text/plain').send('Hello world');
});
```



# Middleware

- Middleware ใน Node.js เป็นฟังก์ชันหรือโค้ดที่ทำงานร่วมกับแอปพลิเคชัน Express.js หรือโดยทั่วไปแล้วกับเซิร์ฟเวอร์ HTTP ใน Node.js เพื่อประมวลผลข้อมูลก่อนที่จะถึงตัวคอนโทรลเลอร์หรือเส้นทาง (route) หลัก
- หน้าที่หลักของ Middleware คือการจัดการข้อมูลและคำขอที่ถูกส่งมายังแอปพลิเคชัน
- โดยสามารถใช้งาน middleware ได้ผ่าน `app.use()`



# Middleware

```
import express, { response } from
"express";

const app = express();
app.use((req, res) => {
 res.send('Hello word1234')
})
app.get("/", (req, res) => {
 res.status(200).type('text/plain')
 .send("Hello world");
});

const PORT = 3000;

app.listen(PORT, () => {
 console.log(`Server running at
http://localhost:${PORT}`);
});
```

- `app.use` → แบบนี้คือ middleware ที่จับทุก request ทุกเส้นทาง
- ถ้า `app.use` ส่ง response แล้ว → route หลัง ๆ จะไม่ถูกเรียก
- ใส่ `next()` เพื่อบอก Express ให้ไปทำงานต่อ
- แต่ ถ้าสลับ `app.get` ไว้บน `app.use` เมื่อมี request เข้ามาด้วย / จะทำให้ `app.get` ทำงาน และหยุด
- นั้นหมายความว่าสามารถกำหนด url โดยใช้ `app.get` หลายอันเพื่อเซ็น url ถ้านอกเหนือจากนั้นทำ `app.use` ได้



# Middleware

```
import express, { response } from "express";

const app = express();
app.use((req, res, next) => {
 console.log("มี request เข้ามา:", req.method, req.url);
 next(); // ไปยัง route ถัดไป
});

app.get("/", (req, res) => {
 res.status(200).type('text/plain').send("Hello world");
});

const PORT = 3000;

app.listen(PORT, () => {
 console.log(`Server running at http://localhost:${PORT}`);
});
```

- req.method บอกว่า คำขอ (Request) นี้ใช้ HTTP Method อะไร
  - เช่น
    - "GET" → ขอข้อมูล
    - "POST" → ส่งข้อมูล
    - "PUT" → แก้ไขข้อมูล
    - "DELETE" → ลบข้อมูล
- req.url บอกว่า path (เส้นทาง URL) ที่ client เรียกเข้ามา คืออะไร
  - เช่น
    - ถ้าเข้า http://localhost:3000/test → req.url จะเป็น "/test"
    - ถ้าเข้า http://localhost:3000/user?id=10 → req.url จะเป็น "/user?id=10"



# การอ่านเนื้อหาจากไฟล์

- `res.sendFile()` เป็น เมธอดของ Response object (`res`) ใน Express.js ใช้สำหรับ ส่งไฟล์ (file) จาก server ไปยัง client โดยตรง
- เหมาะกับกรณีต้องการให้ผู้ใช้งานดาวน์โหลดหรือแสดงผลไฟล์ เช่น .html, .jpg, .css, .pdf ฯลฯ
- เริ่มต้นจะต้อง

```
import path from "node:path";
import { fileURLToPath } from "node:url";
```

- 



# การอ่านเนื้อหาจากไฟล์

- import path from "node:path";
  - path เป็น โมดูลของ Node.js ที่ใช้สำหรับ จัดการเส้นทางไฟล์/โฟลเดอร์
  - ช่วยต่อ path, แยกชื่อไฟล์, หาที่อยู่โฟลเดอร์ ฯลฯ โดยไม่ต้องกังวลว่าจะอยู่ Windows (\) หรือ Linux (/)

```
import path from "node:path";

console.log(path.join("public", "images", "logo.png"));
// public/images/logo.png (Linux / Mac)
// public\images\logo.png (Windows)

console.log(path.extname("index.html"));
//.html (หาค่าส่วนนามสกุลไฟล์)

console.log(path.basename("/user/test/index.html"));
//index.html (หาชื่อไฟล์)
```



# การอ่านเนื้อหาจากไฟล์

- `import { fileURLToPath } from "node:url";`
  - `url` เป็นอีกโมดูลของ Node.js สำหรับจัดการ URL
  - `fileURLToPath` เป็นฟังก์ชันที่ใช้ แปลง URL แบบ `file:///` ให้กลายเป็น path ปกติของระบบไฟล์

```
import { fileURLToPath } from "node:url";

// Windows
const fileUrl = new URL("file:///C:/Users/Lenovo/Desktop/test/test/index.js");
console.log(fileURLToPath(fileUrl));
// C:\Users\Lenovo\Desktop\test\test\index.js
```



# การอ่านเนื้อหาจากไฟล์

- ใน ES Module (ESM) จะไม่มีตัวแปรพิเศษ `__dirname` และ `__filename` แบบ CommonJS จึงต้องสร้างเองแบบนี้

```
import path from "node:path";
import { fileURLToPath } from "node:url";

const __filename = fileURLToPath(import.meta.url);
// path เต็มของไฟล์ปัจจุบัน

const __dirname = path.dirname(__filename);
// path ของโฟลเดอร์ที่ไฟล์ปัจจุบันอยู่
```



# การอ่านเนื้อหาจากไฟล์

ไฟล์ index.js

```
import express from "express";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();

app.get("/", (req, res) => {
 res.status(200)
 res.type('text/html')
 res.sendFile(path.join(__dirname, "html", "index.html"));
 // ใช้ __dirname หาดำแหน่งไฟล์
});

const PORT = 3000;

app.listen(PORT, () => {
 console.log(`Server running at http://localhost:${PORT}`);
});
```

ไฟล์ index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-
width, initial-scale=1.0">
 <title>Document</title>
</head>
<body>
 <p id="idCard"></p>
 <script >
 let p = document.getElementById('idCard')
 p.innerText = "hello"
 </script>
</body>
</html>
```

```
import express from "express";
import path from "node:path";
import { fileURLToPath } from "node:url";
import fs from "node:fs";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();

app.get("/", (req, res) => {
 const filePath = path.join(__dirname, "html", "index.html")
 // อ่านไฟล์เองก่อน
 fs.access(filePath, fs.constants.F_OK, (err) => {
 if (err) {
 // กรณีไฟล์ไม่พบ
 res.status(500).type("text/plain; charset=utf-8").send("อ่านไฟล์ไม่ได้");
 } else {
 // ถ้ามีไฟล์ → ส่งด้วย sendFile
 res.status(200).type("text/html").sendFile(filePath);
 }
 });
});

const PORT = 3000;
app.listen(PORT, () => {
 console.log(`Server running at http://localhost:${PORT}`);
});
```

# การอ่านเนื้อหาจากไฟล์

- import fs from "node:fs"
  - fs = ย่อมาจาก File System เป็น built-in module ของ Node.js ใช้จัดการไฟล์และโฟลเดอร์ (อ่าน, เขียน, สร้าง, ลบ, เช็คสถานะ ฯลฯ)
- ใช้ fs.access เพื่อตรวจสอบว่าไฟล์/โฟลเดอร์มีอยู่จริงหรือไม่
  - มีโครงสร้าง fs.access(path, mode, callback)
  - path
    - ตำแหน่งไฟล์หรือโฟลเดอร์ที่ต้องการตรวจสอบ
    - เช่น "output.txt" หรือใช้ path.join(\_\_dirname, "output.txt")
  - mode
    - เป็น flag ที่กำหนดสิ่งที่เราจะตรวจสอบ
    - นิยมใช้จาก fs.constants เช่น
      - fs.constants.F\_OK → ตรวจสอบว่ามีไฟล์อยู่จริงหรือไม่ (Existence check)
      - fs.constants.R\_OK → ตรวจสอบสิทธิ์อ่านได้หรือไม่ (Readable)
      - fs.constants.W\_OK → ตรวจสอบสิทธิ์เขียนได้หรือไม่ (Writable)
      - fs.constants.X\_OK → ตรวจสอบสิทธิ์ execute ได้หรือไม่ (สำหรับไฟล์ executable)
  - callback → ฟังก์ชัน (err) ที่ถูกเรียกเมื่อเช็คเสร็จ
    - ถ้าไฟล์ ไม่เข้าเงื่อนไข → err จะเป็น object
    - ถ้าไฟล์ เข้าเงื่อนไข → err = null

# ใบงาน

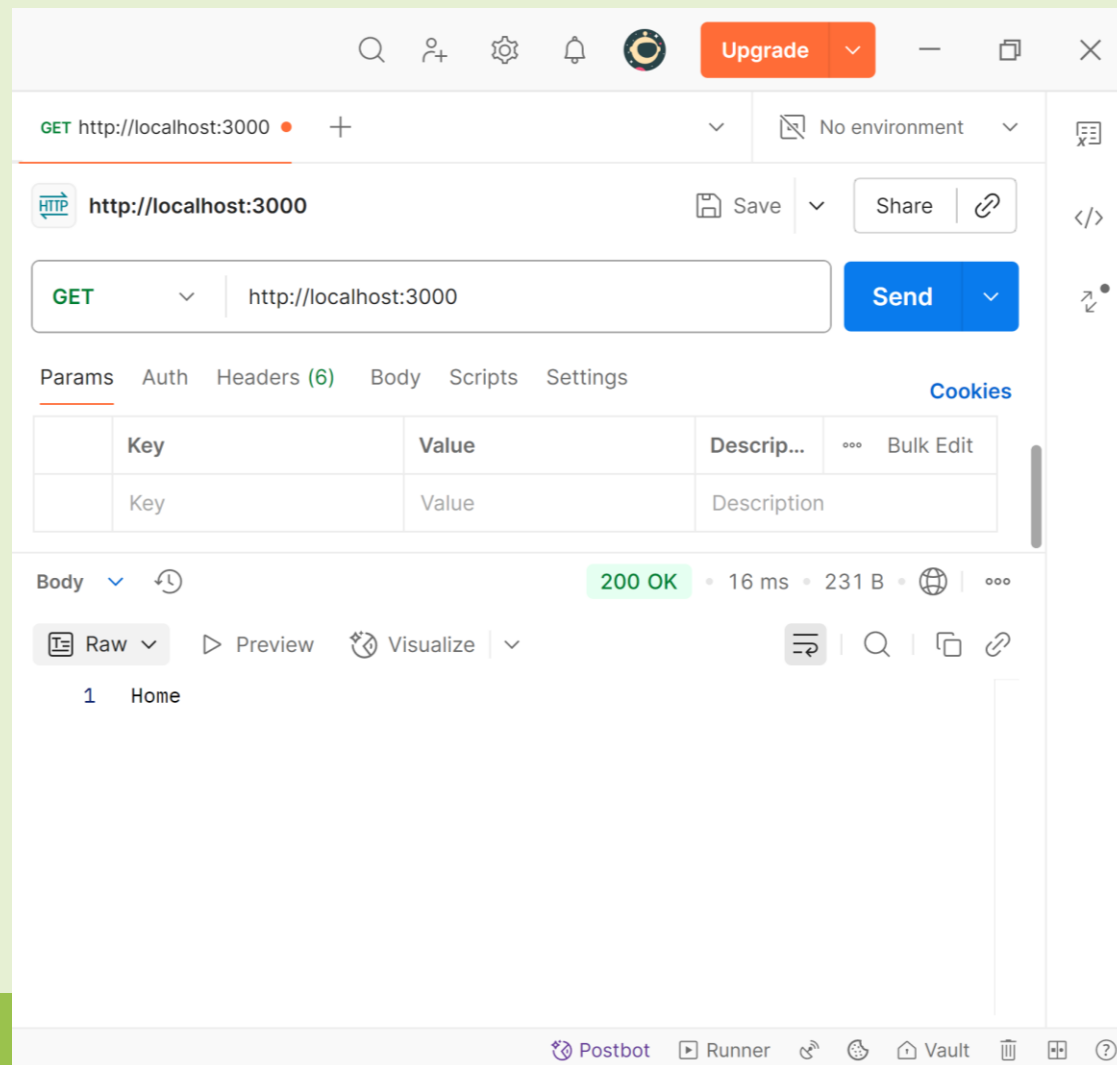
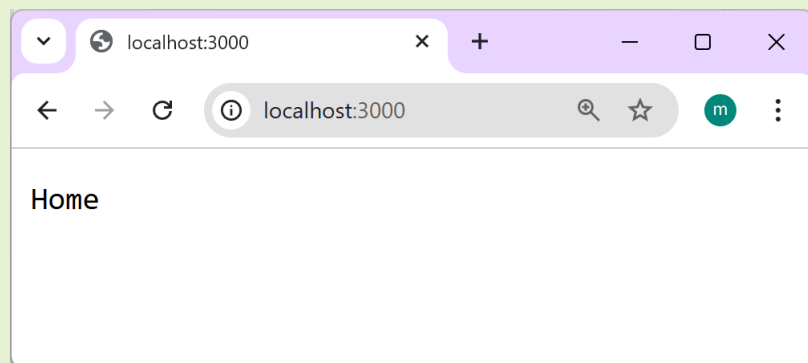
- 1) สร้าง ใช้ Express.js สร้าง Web Server + เส้นทางพื้นฐาน
- สร้างไฟล์เตอร์ no1 สร้างไฟล์ app.js ที่รองรับ 3 เส้นทาง
  - GET / → ตอบ text/plain ว่า Home
  - GET /about → ตอบ text/html (มี <h1>About</h1>)
  - เส้นทางอื่น ๆ → ตอบ 404 Not Found
- ทดสอบ:
- เปิด <http://localhost:3000/>, <http://localhost:3000/about>, <http://localhost:3000/xyz>



# ใบงาน

- ทดสอบ:

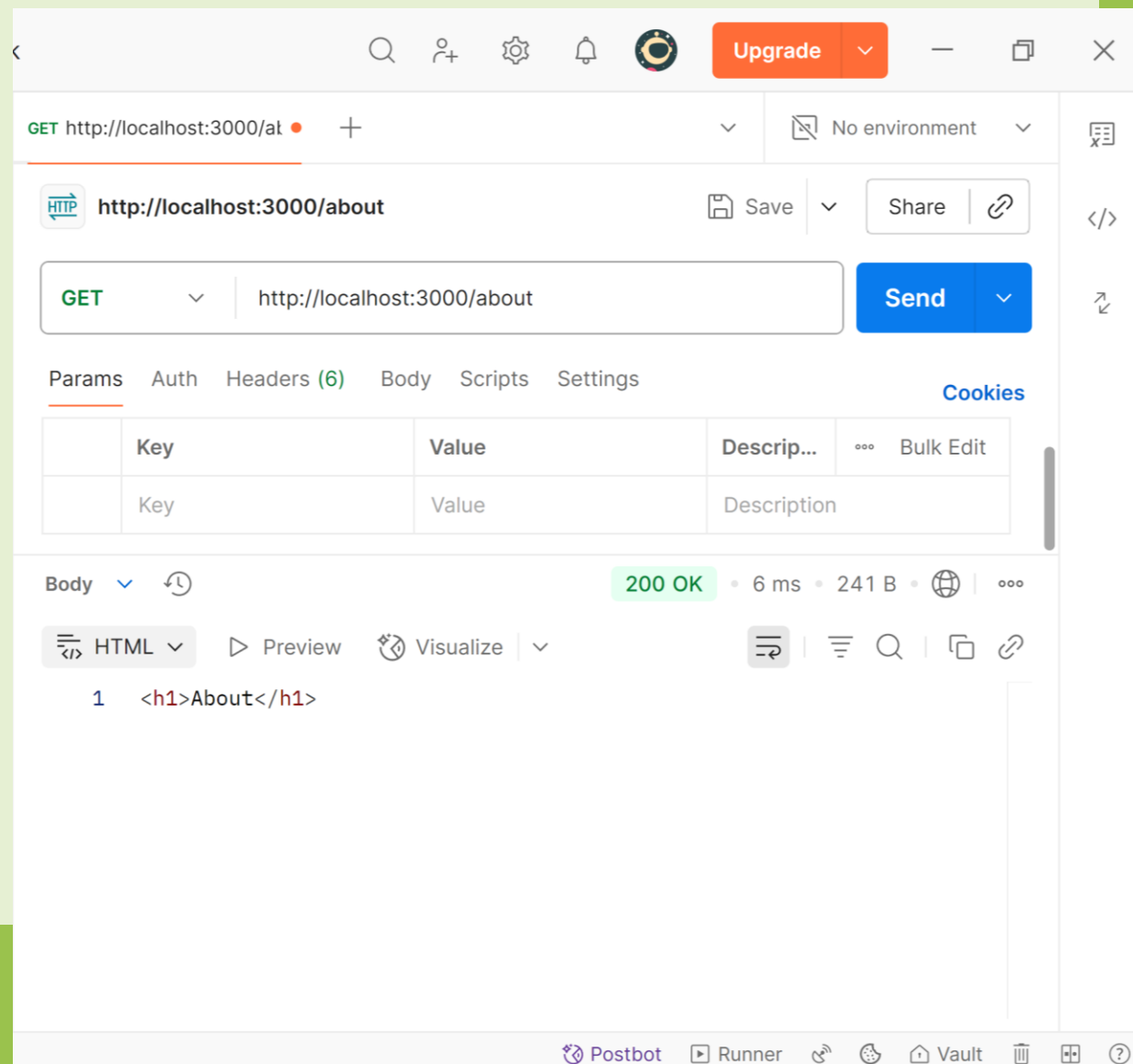
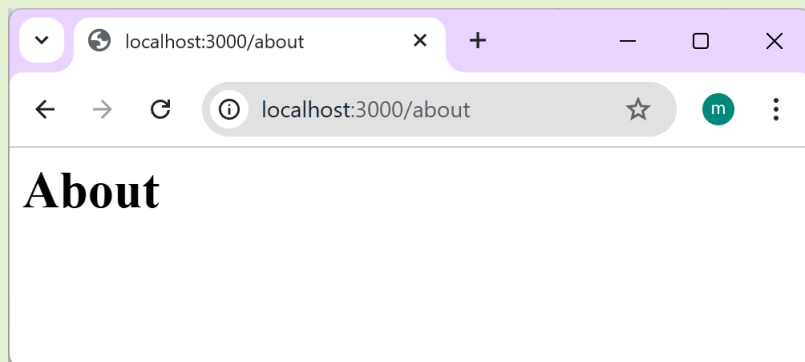
เปิด `http://localhost:3000/`



# ใบงาน

- ทดสอบ:

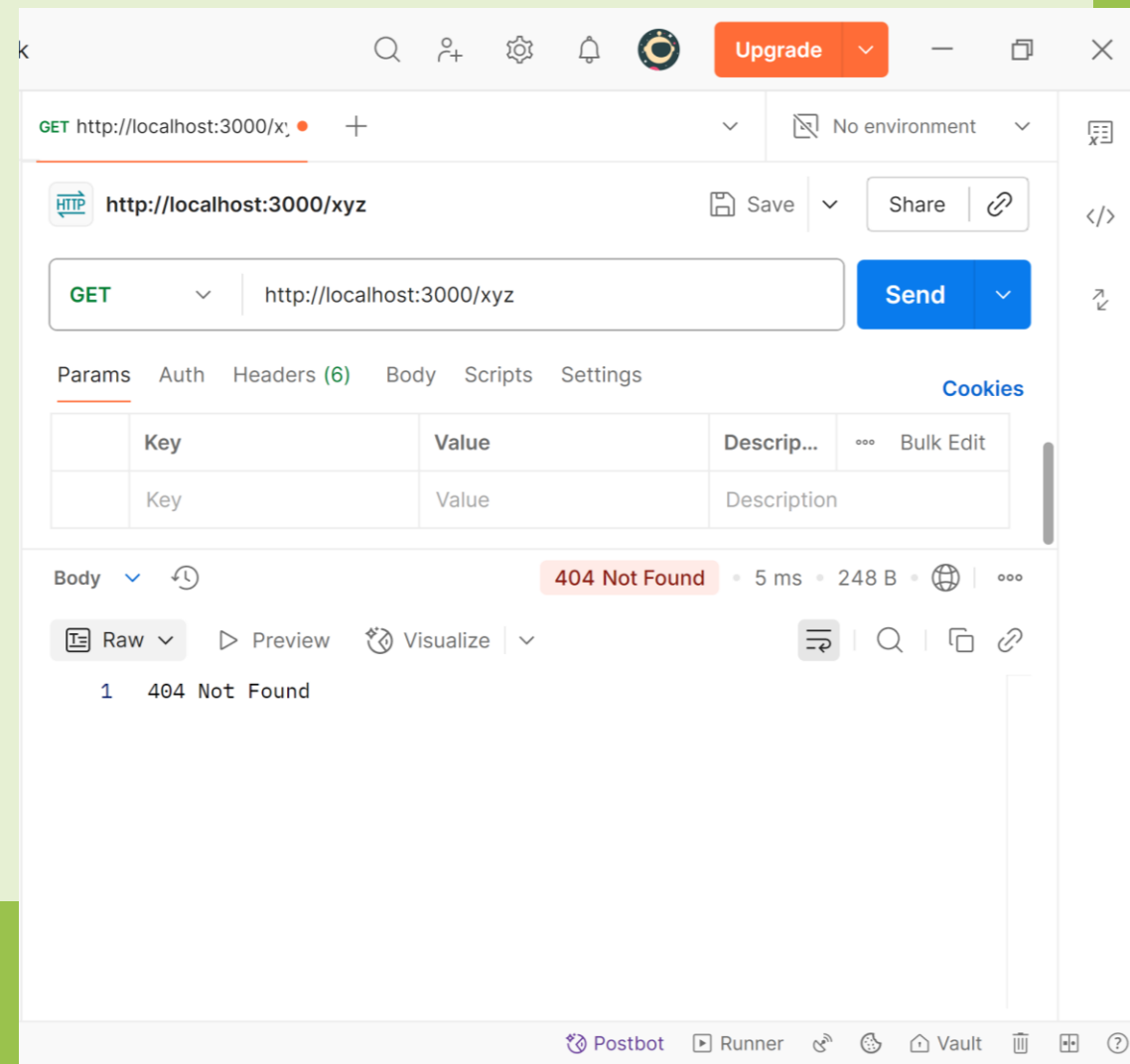
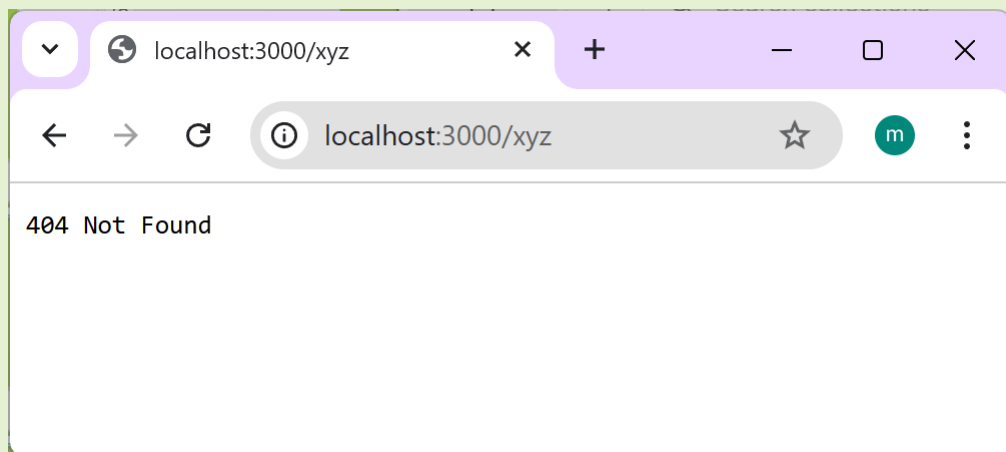
http://localhost:3000/about



# ใบงาน

- ทดสอบ:

http://localhost:3000/xyz



# ใบงาน

- 2. ให้สร้างโครงสร้างโปรเจกต์ดังนี้

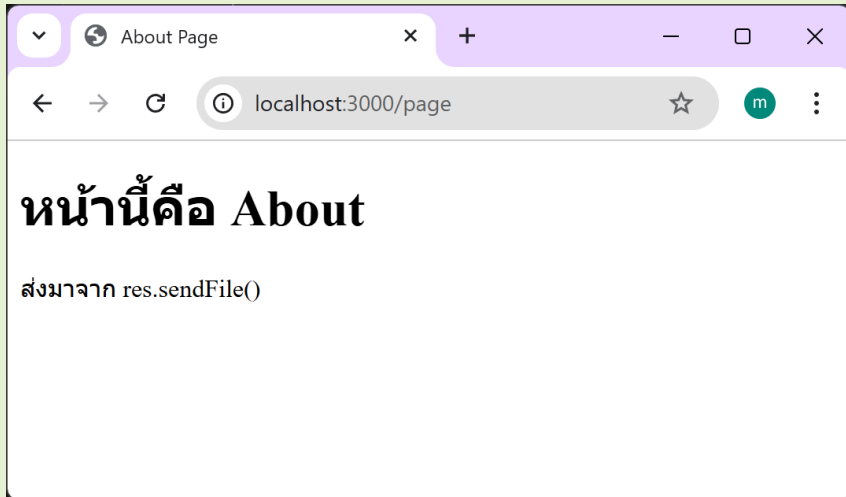
```
no2/
├── index.js
└── html/
 └── about.html
```

- อ่านไฟล์ HTML เองด้วย fs.readFile
- ใช้ Express.js สร้าง Web Server ให้รองรับ GET /page แล้วอ่านไฟล์ about.html (อยู่โฟลเดอร์ html) มาตอบกลับเงื่อนไข: Content-Type: text/html; charset=utf-8 กรณีไฟล์ไม่พบ ให้ตอบ 500 พร้อมข้อความ “อ่านไฟล์ไม่ได้”
- 



# ใบงาน

- ทดสอบ: เข้า `http://localhost:3000/page`



- ทดสอบ: เข้า `http://localhost:3000/page`  
แก้ไขชื่อไฟล์ให้อ่านไม่เจอ

